



Politecnico
di Torino

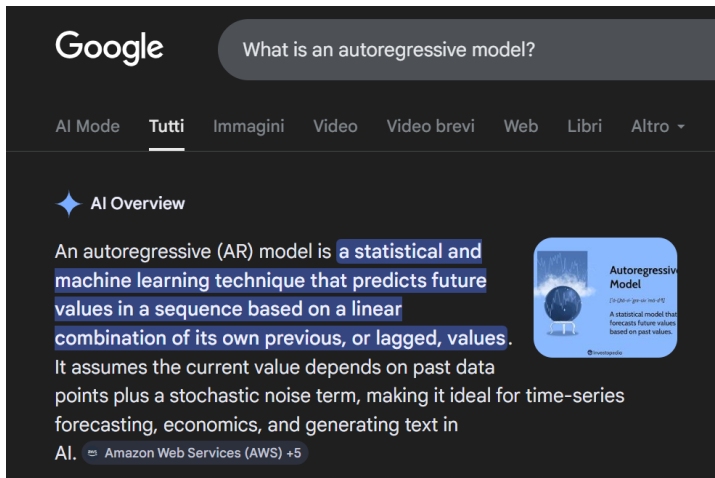
Generative Artificial Intelligence for graphics and multimedia

Transformers and auto-regressive
models

Prof. Lia Morra



Guess the common element



Response generated by Gemini

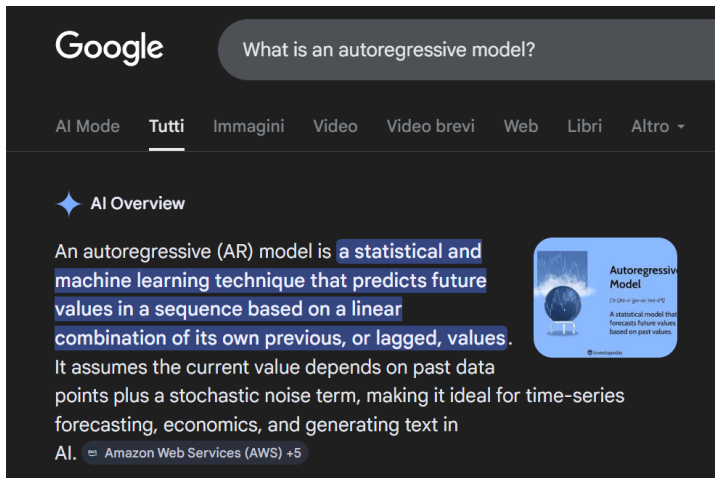


Images generated by VAR



Video generated by NOVA
https://bitterdhg.github.io/NOVA_page/

Guess the common element



Response generated by Gemini



Images generated by VAR

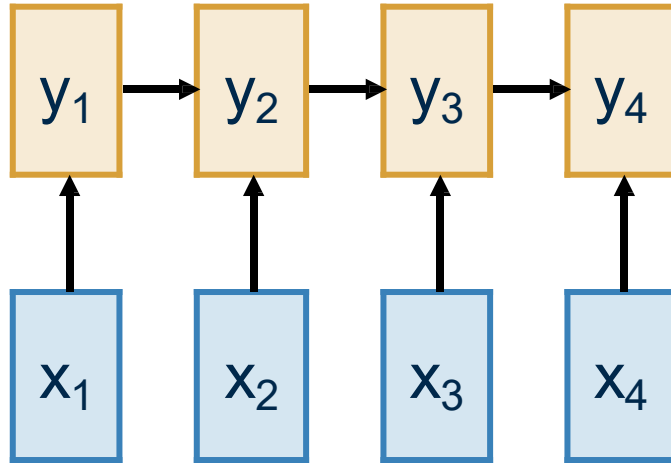


Video generated by NOVA
https://bitterdhg.github.io/NOVA_page/

All these contents are generated by neural networks that share the same underlying **auto-regressive** and **attention**-based DNA.

Recall: three ways of processing sequences

Recurrent Neural Network



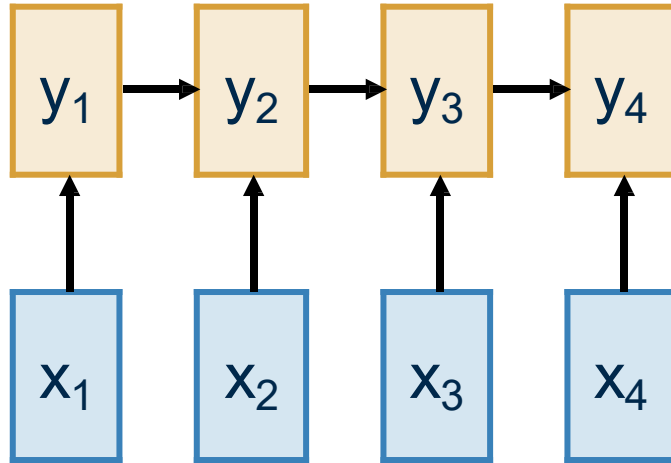
Works on **1D ordered sequences**

(+) Theoretically good at long sequences: $O(N)$ compute and memory for a sequence of length N

(-) Not parallelizable. Need to compute hidden states sequentially

Recall: three ways of processing sequences

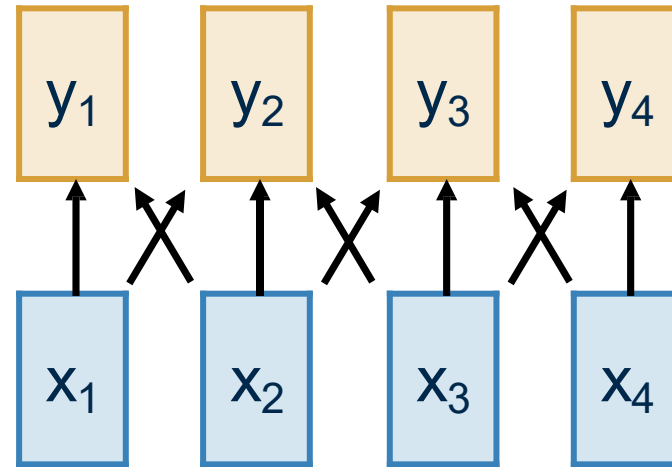
Recurrent Neural Network



Works on **1D ordered sequences**

(+) Theoretically good at long sequences: $O(N)$ compute and memory for a sequence of length N (-)
) Not parallelizable. Need to compute hidden states sequentially

Convolution

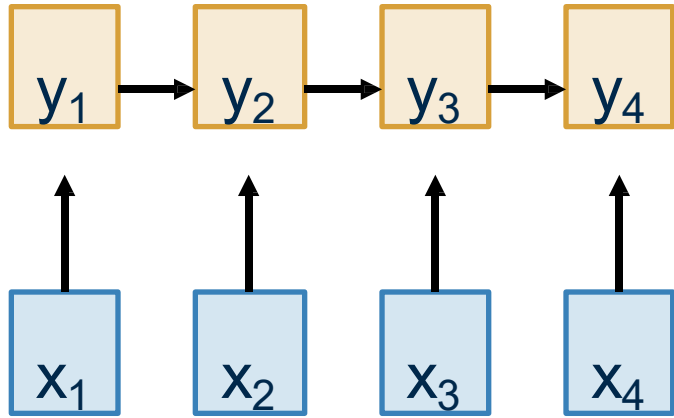


Works on **N-dimensional grids**

(-) Bad for long sequences: need to stack many layers to build up large receptive fields
(+) Parallelizable, outputs can be computed in parallel

Recall: three ways of processing sequences

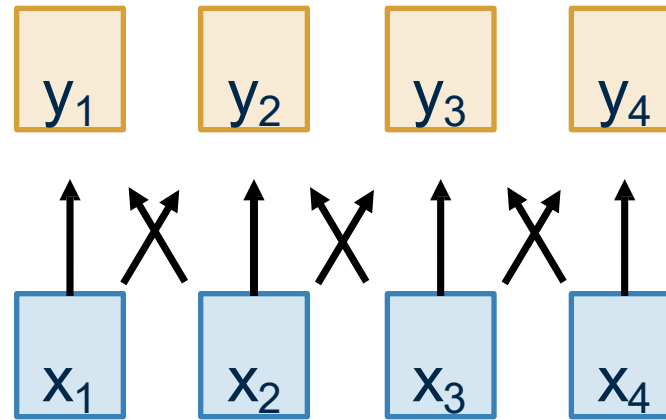
Recurrent Neural Network



Works on **1D ordered sequences**

- (+) Theoretically good at long sequences: $O(N)$ compute and memory for a sequence of length N
- (-) Not parallelizable. Need to compute hidden states sequentially

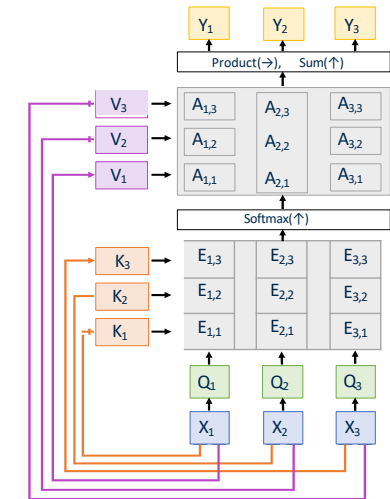
Convolution



Works on **N-dimensional grids**

- (-) Bad for long sequences: need to stack many layers to build up large receptive fields
- (+) Parallelizable, outputs can be computed in parallel

Self-Attention



Works on **sets of vectors**

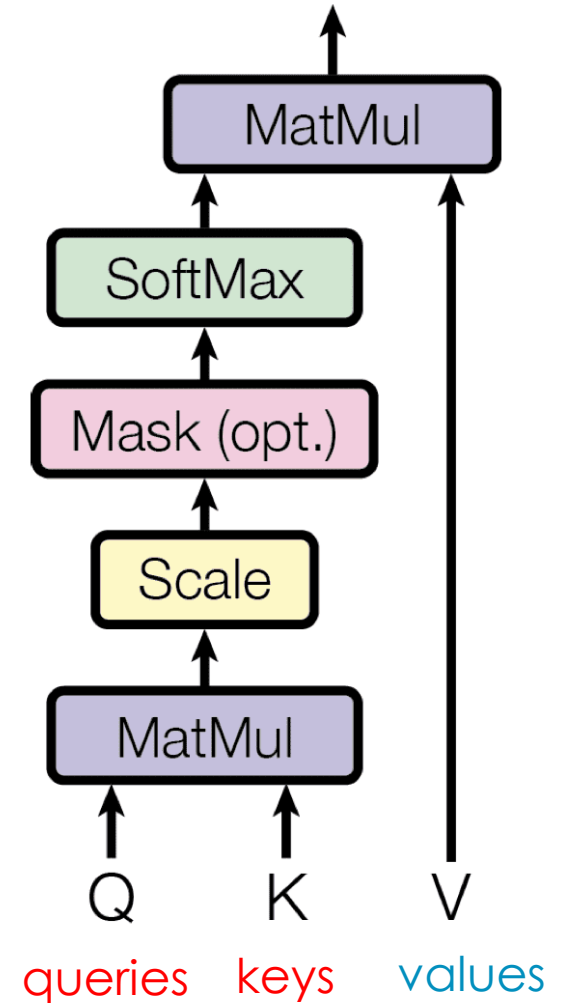
- (+) Great for long sequences; each output depends directly on all inputs
- (+) Highly parallel, it's just 4 matmuls
- (-) Expensive: $O(N^2)$ compute, $O(N)$ memory for sequence of length N

Transformers

- Transformers process sequences of tokens – e.g., words – in parallel
- The building block is the scale dot-product attention
- Attention measure how “compatible” queries and keys are

$$\text{Attention}(\textcolor{red}{Q}, \textcolor{red}{K}, \textcolor{teal}{V}) = \text{softmax}\left(\frac{\textcolor{red}{QK}}{\sqrt{d_k}}\right) \textcolor{teal}{V}$$

[Vaswani et. al., 2017. Attention is all you need]

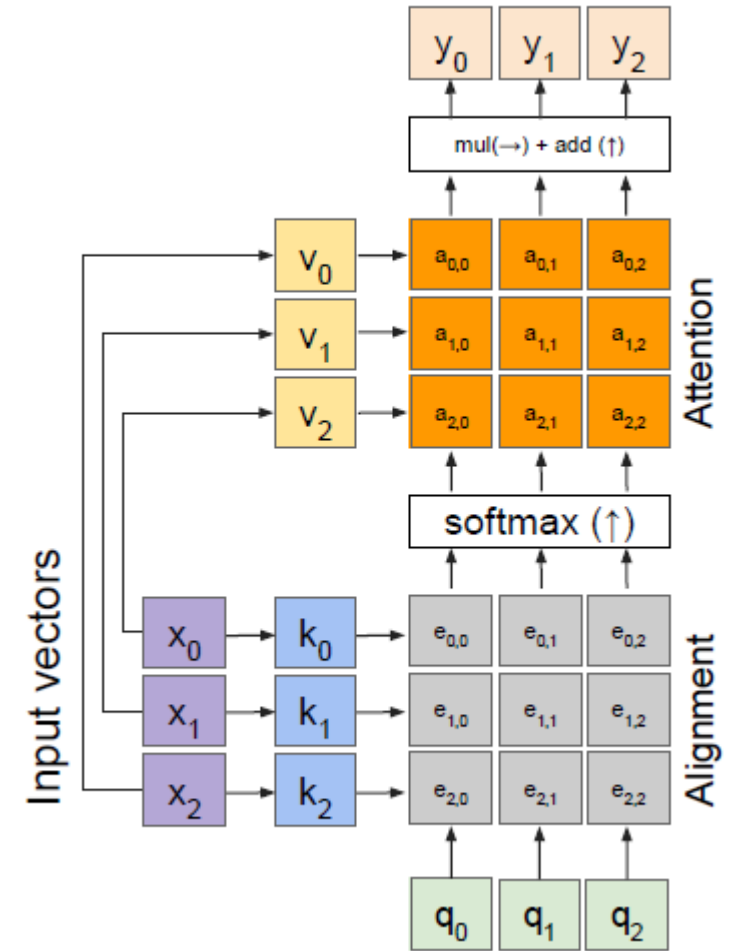


Transformers

- **Queries** and **keys** are vectors of size d_k
- Key, values and query are linearly projected by multiplying with learnable weight matrices W^Q , W^K , W^V
- Softmax ensures that all weights sum to 1

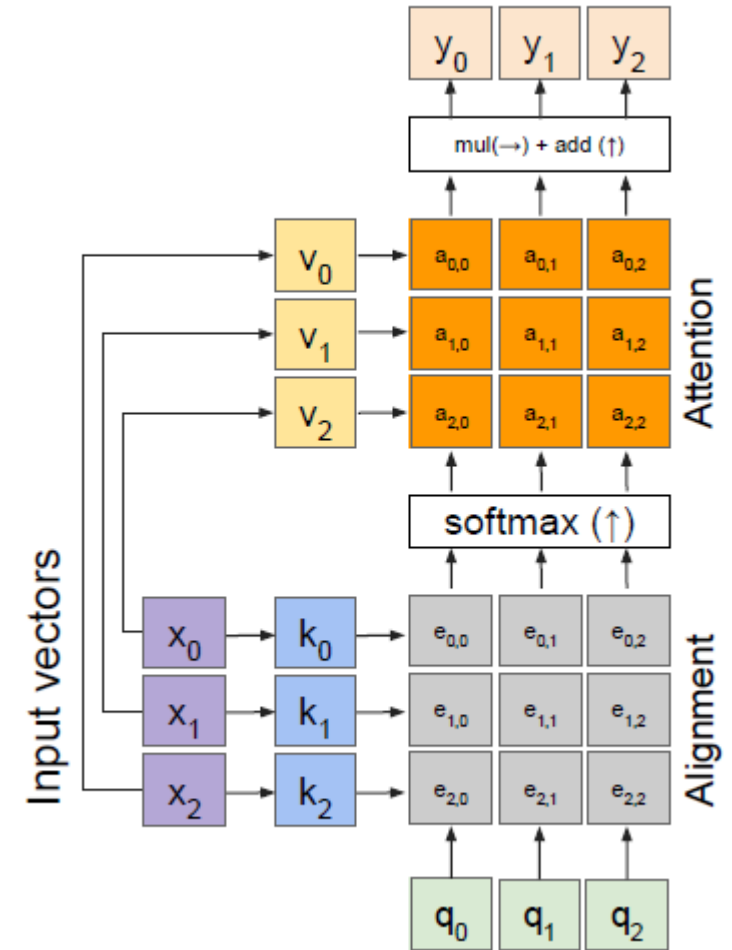
$$\text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}$$

[Vaswani et. al., 2017. Attention is all you need]



Attention

- We distinguish between **self-attention**:
 - A vector of inputs $x^{<1>}$, $x^{<2>}$, ..., $x^{<t>}$ can serve simultaneously as key, query and vectors
- and **cross-attention**:
 - Queries and keys are extracted from different sequences
- We may or may not “look” into the future



Attention Layer – deep dive

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

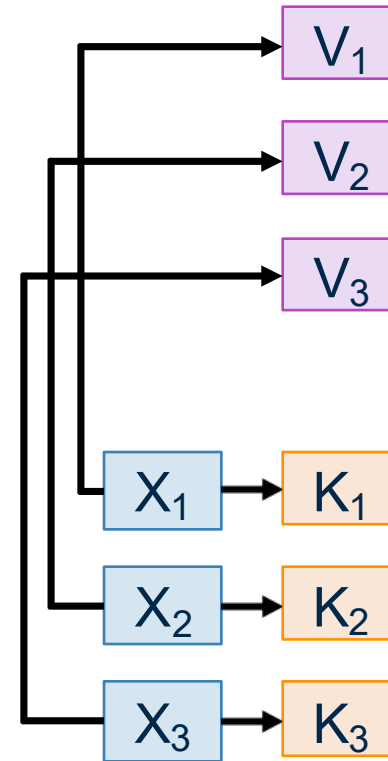
Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]



Attention Layer – deep dive

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

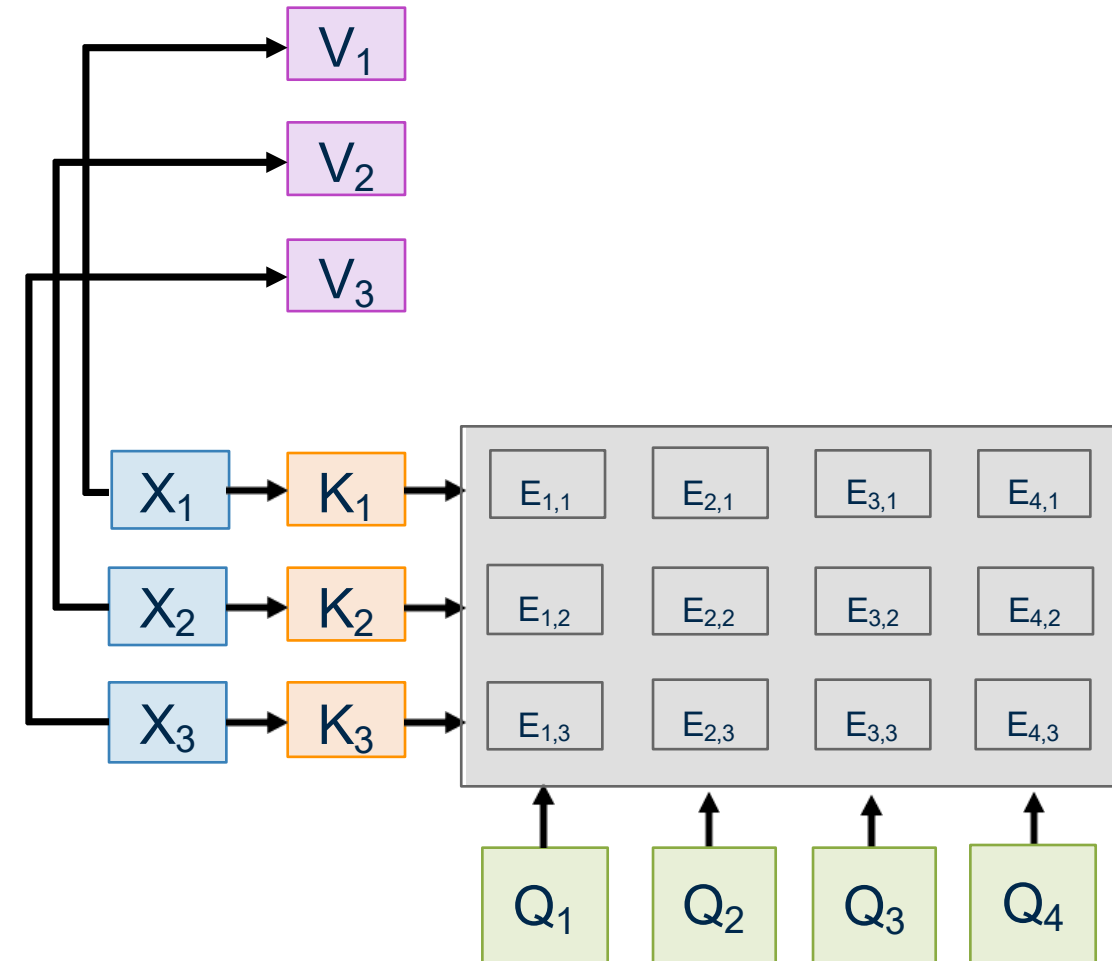
Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]
 $E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

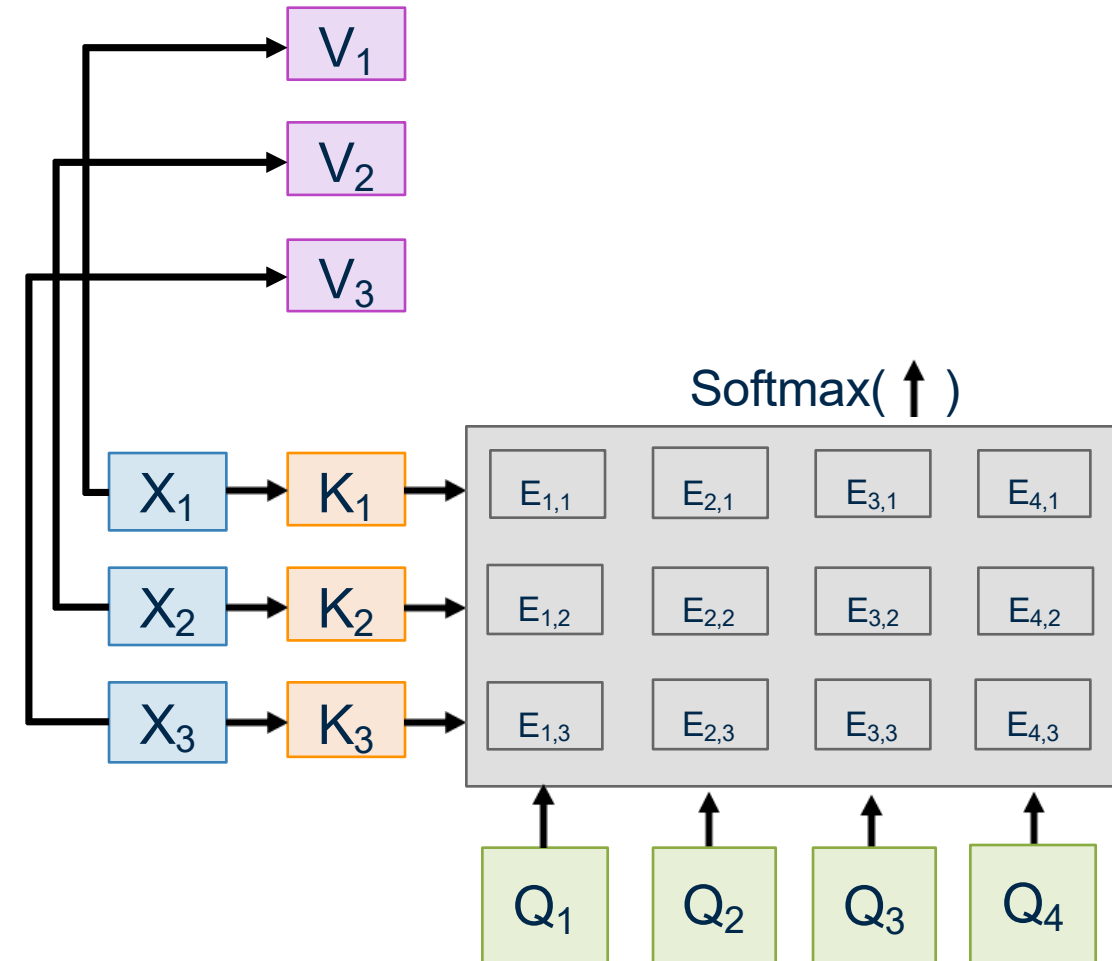
Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]
 $E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

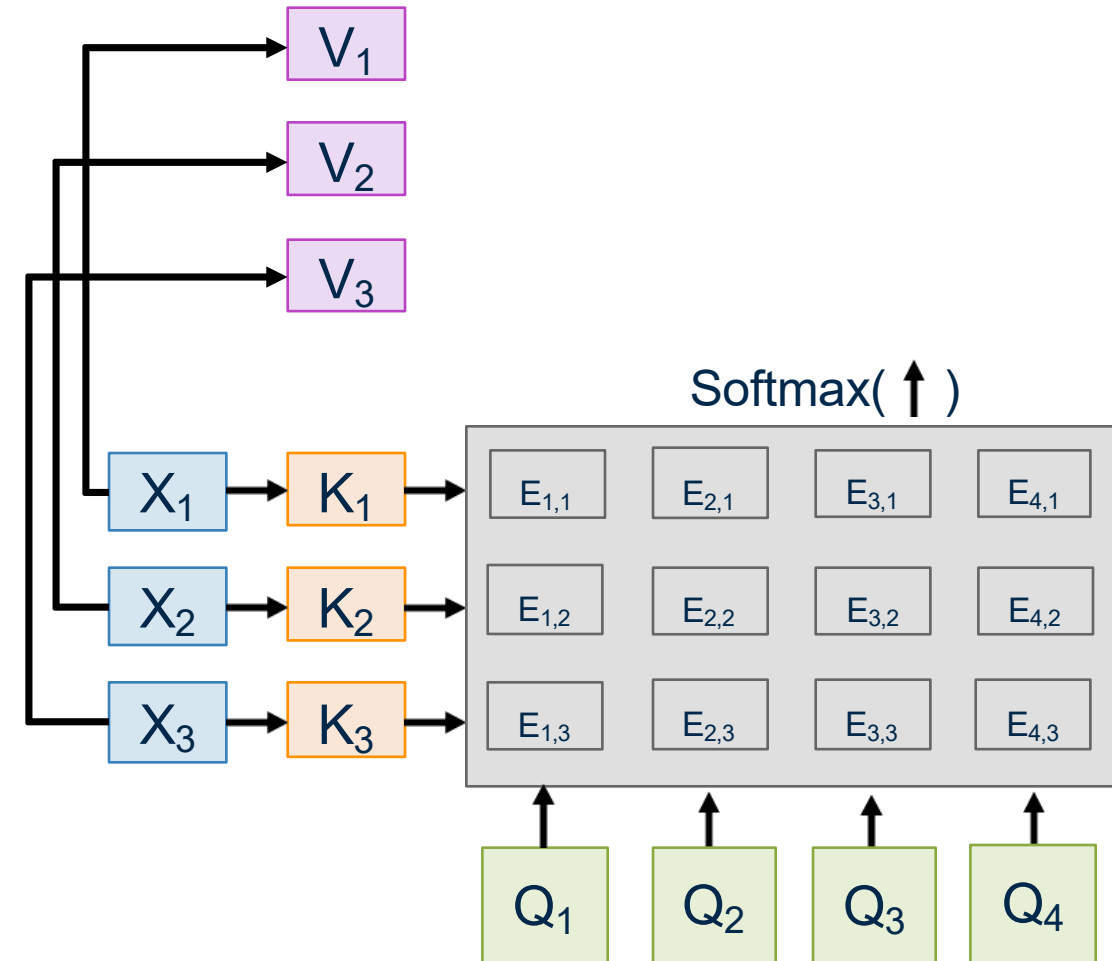
Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]
 $E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Softmax normalizes each column:
each **query** predicts a distribution
over the **keys**



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

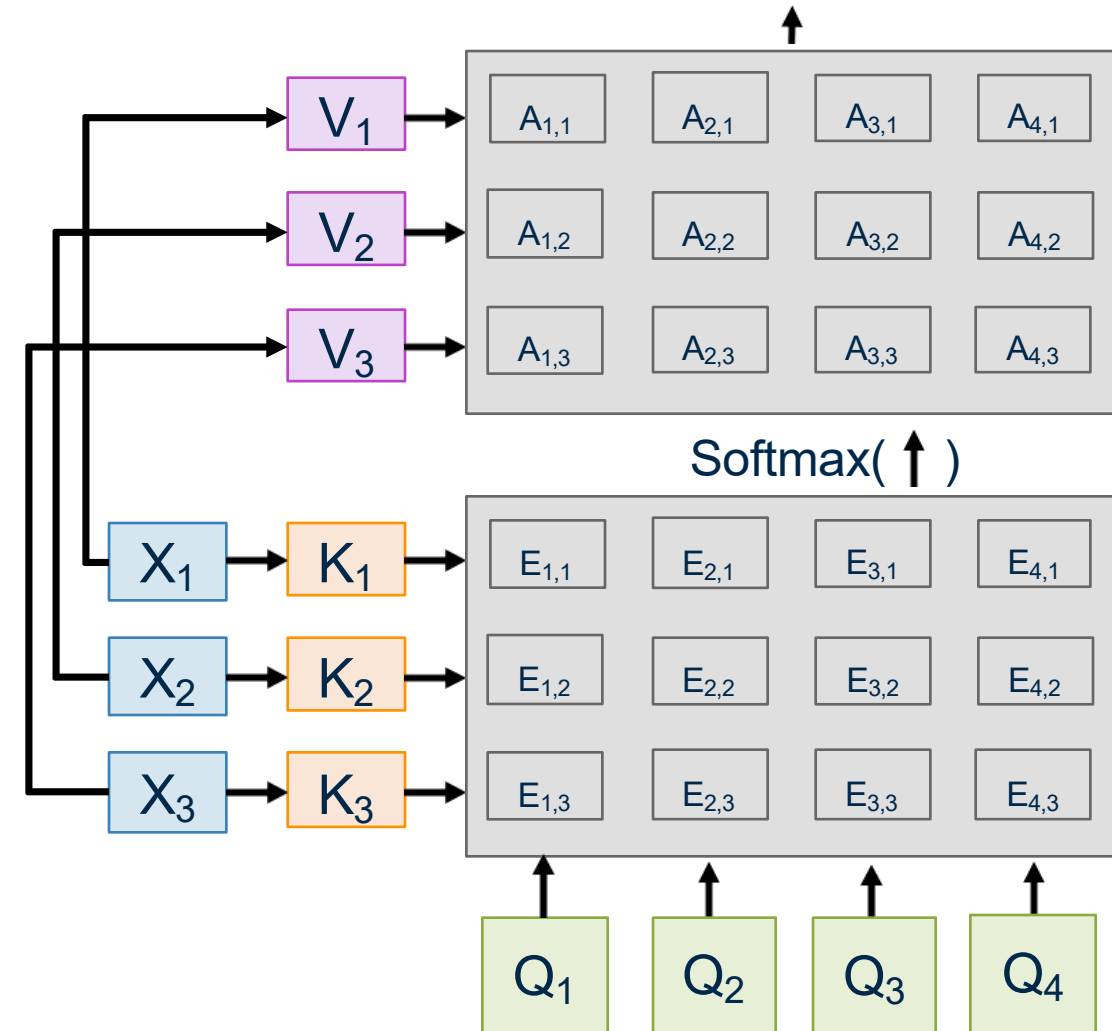
Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]
 $E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]



Attention Layer

Inputs:

Query vector: $\mathbf{Q}$ [$N_Q \times D_Q$]

Data vectors: $\mathbf{X}$ [$N_X \times D_X$]

Key matrix: $\mathbf{W}_K$ [$D_X \times D_Q$]

Value matrix: $\mathbf{W}_V$ [$D_X \times D_V$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N_X \times D_Q$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N_X \times D_V$]

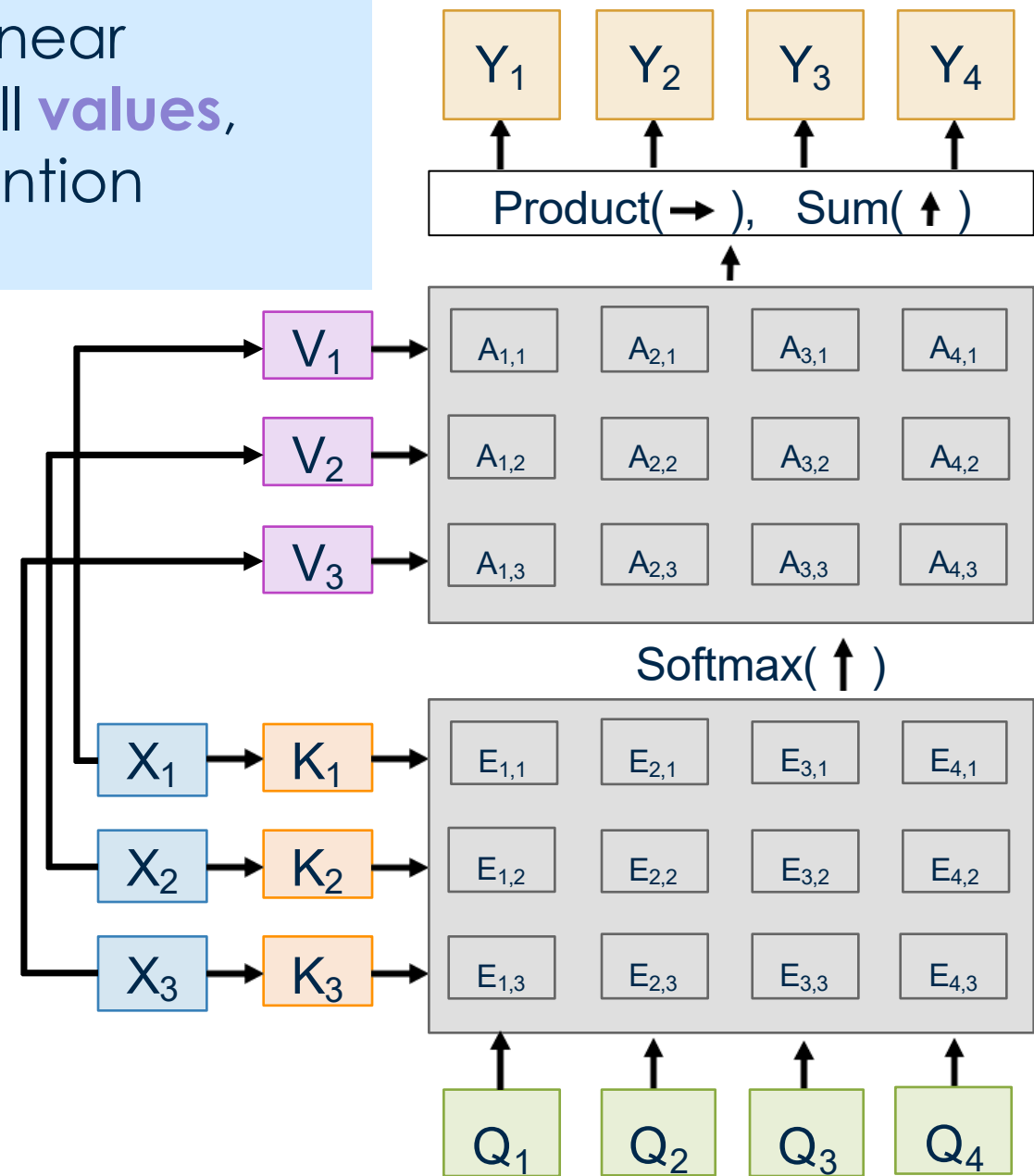
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{D_Q}$ [$N_Q \times N_X$]
 $E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{D_Q}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$ [$N_Q \times N_X$]

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N_Q \times D_V$]

$$Y_i = \sum_j A_{ij} V_j$$

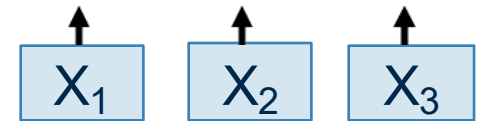
Each **output** is a linear combination of all **values**, weighted by attention weights



Self-attention layer – deep dive

Inputs

Data vectors: X [$N_x \times d_k$]



Self-attention layer

From each **input**: compute a **query**, **key**, and **value** vector

Inputs:

Data vectors: $\mathbf{X}$ $[N \times d_k]$

Weights:

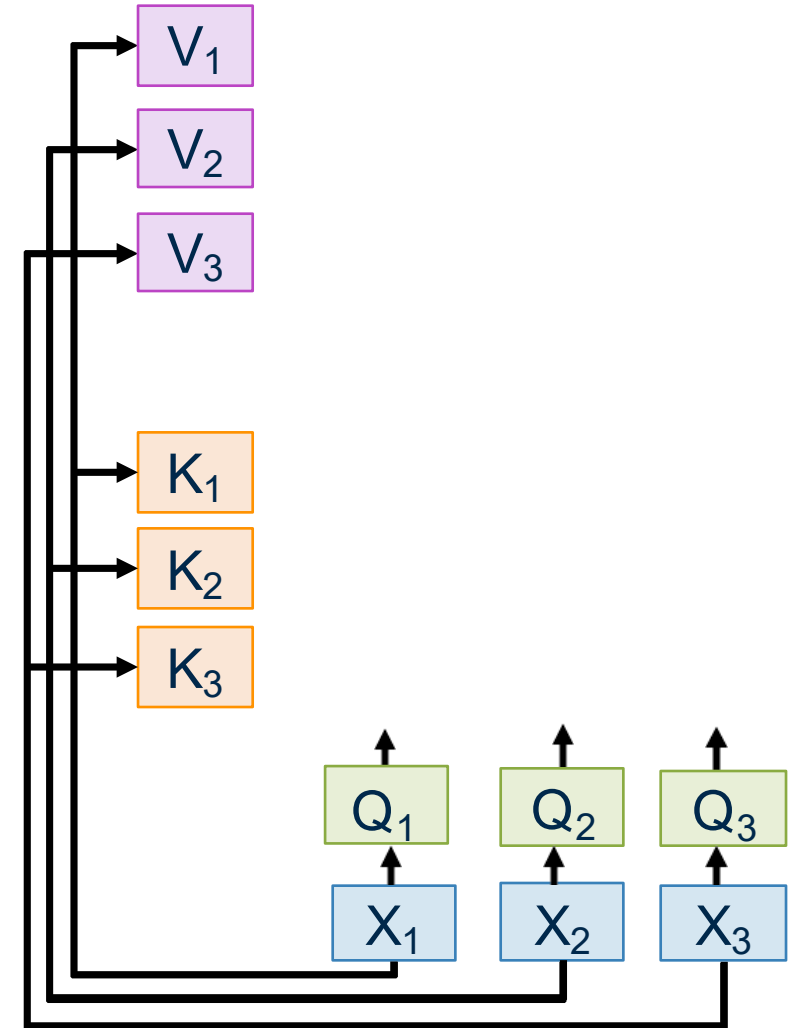
Key matrix: $\mathbf{W}_k$ $[d_k \times d_{out}]$

Query matrix: $\mathbf{W}_q$ $[d_k \times d_{out}]$

Value matrix: $\mathbf{W}_v$ $[d_k \times d_{out}]$

Shape of the weight matrices is usually the same

Each **input** produces one **output**, which is a mix of information from all **inputs**



Self-attention layer

Inputs:

Data vectors: $\mathbf{X}$ $[N \times d_k]$

Weights:

Key matrix: $\mathbf{W}_k$ $[d_k \times d_{out}]$

Query matrix: $\mathbf{W}_q$ $[d_k \times d_{out}]$

Value matrix: $\mathbf{W}_v$ $[d_k \times d_{out}]$

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_k$ $[N \times d_{out}]$

Values: $\mathbf{V} = \mathbf{XW}_v$ $[N \times d_{out}]$

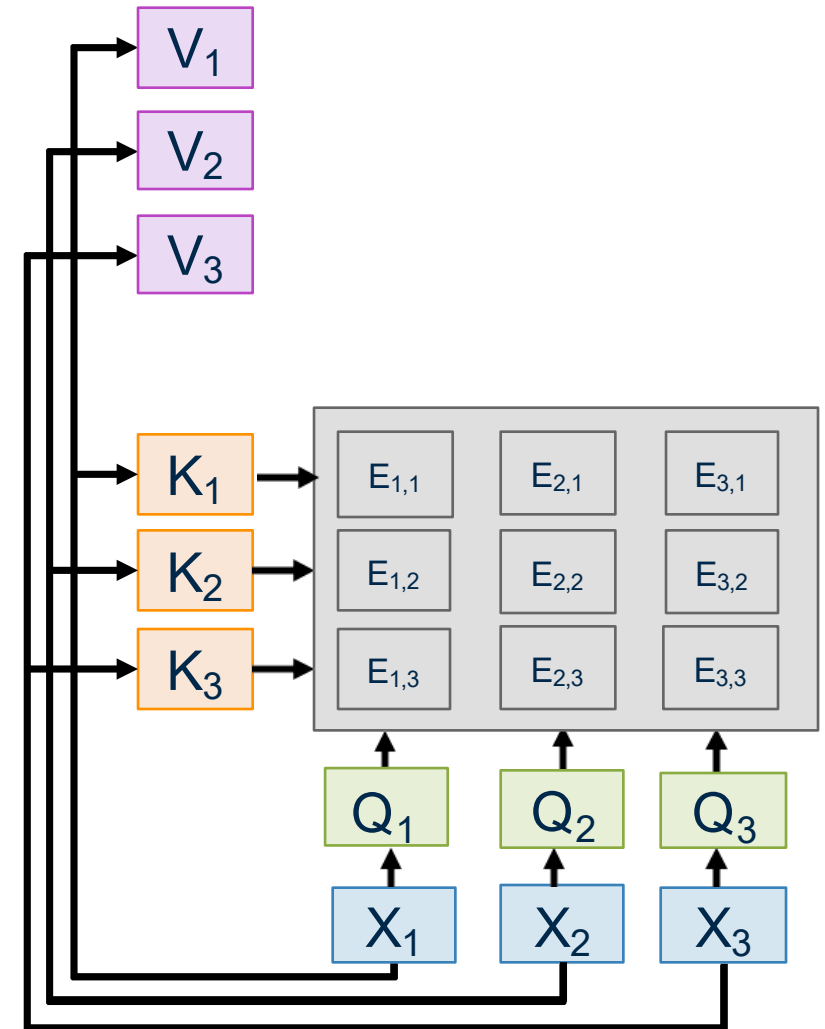
Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{d_k}$ $[N \times N]$

$$E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{d_k}$$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$

Output vector: $\mathbf{Y} = \mathbf{AV}$ $[N \times d_{out}]$

$$\mathbf{Y}_i = \sum_j \mathbf{A}_{ij} \mathbf{V}_j$$



Self-attention layer

Inputs:

Data vectors: $\mathbf{X}$ $[N \times d_k]$

Weights:

Key matrix: $\mathbf{W}_K$ $[d_k \times d_{out}]$

Query matrix: $\mathbf{W}_Q$ $[d_k \times d_{out}]$

Value matrix: $\mathbf{W}_V$ $[d_k \times d_{out}]$

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[N \times d_{out}]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[N \times d_{out}]$

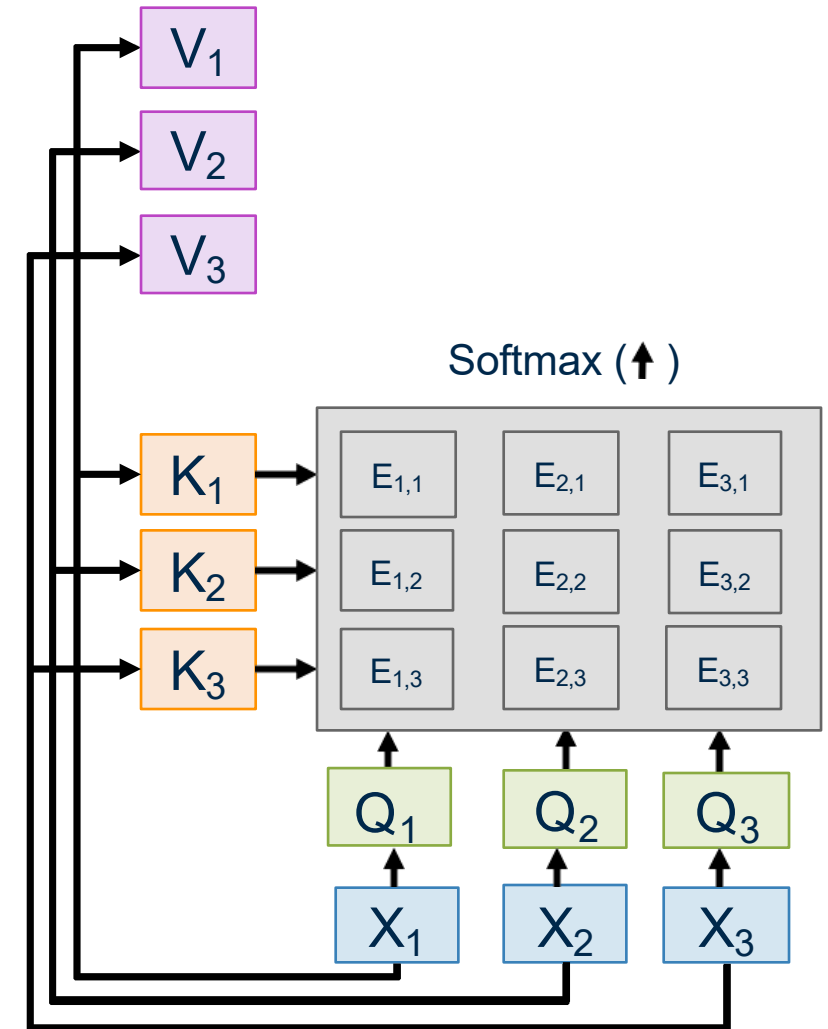
Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[N \times d_{out}]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{d_k}$ $[N \times N]$
 $E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{d_k}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$

Output vector: $\mathbf{Y} = \mathbf{AV}$ $[N \times d_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$



Self-attention layer

Inputs:

Data vectors: $\mathbf{X}$ $[N \times d_k]$

Weights:

Key matrix: $\mathbf{W}_K$ $[d_k \times d_{out}]$

Query matrix: $\mathbf{W}_Q$ $[d_k \times d_{out}]$

Value matrix: $\mathbf{W}_V$ $[d_k \times d_{out}]$

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[N \times d_{out}]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[N \times d_{out}]$

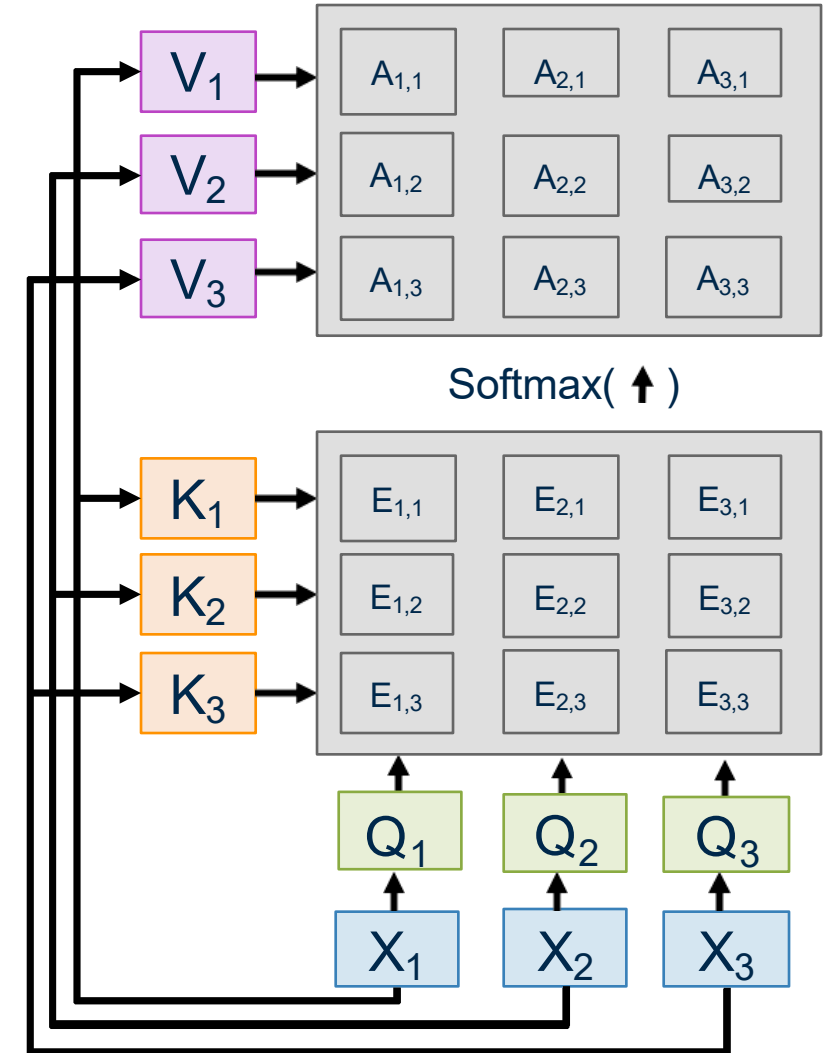
Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[N \times d_{out}]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{d_k}$ $[N \times N]$
 $E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{d_k}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$

Output vector: $\mathbf{Y} = \mathbf{AV}$ $[N \times d_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$



Self-attention layer

Each **output** is a linear combination of all **values**.

Inputs:

Data vectors: $\mathbf{X}$ [$N \times d_k$]

Weights:

Key matrix: $\mathbf{W}_K$ [$d_k \times d_{out}$]

Query matrix: $\mathbf{W}_Q$ [$d_k \times d_{out}$]

Value matrix: $\mathbf{W}_V$ [$d_k \times d_{out}$]

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ [$N \times d_{out}$]

Values: $\mathbf{V} = \mathbf{XW}_V$ [$N \times d_{out}$]

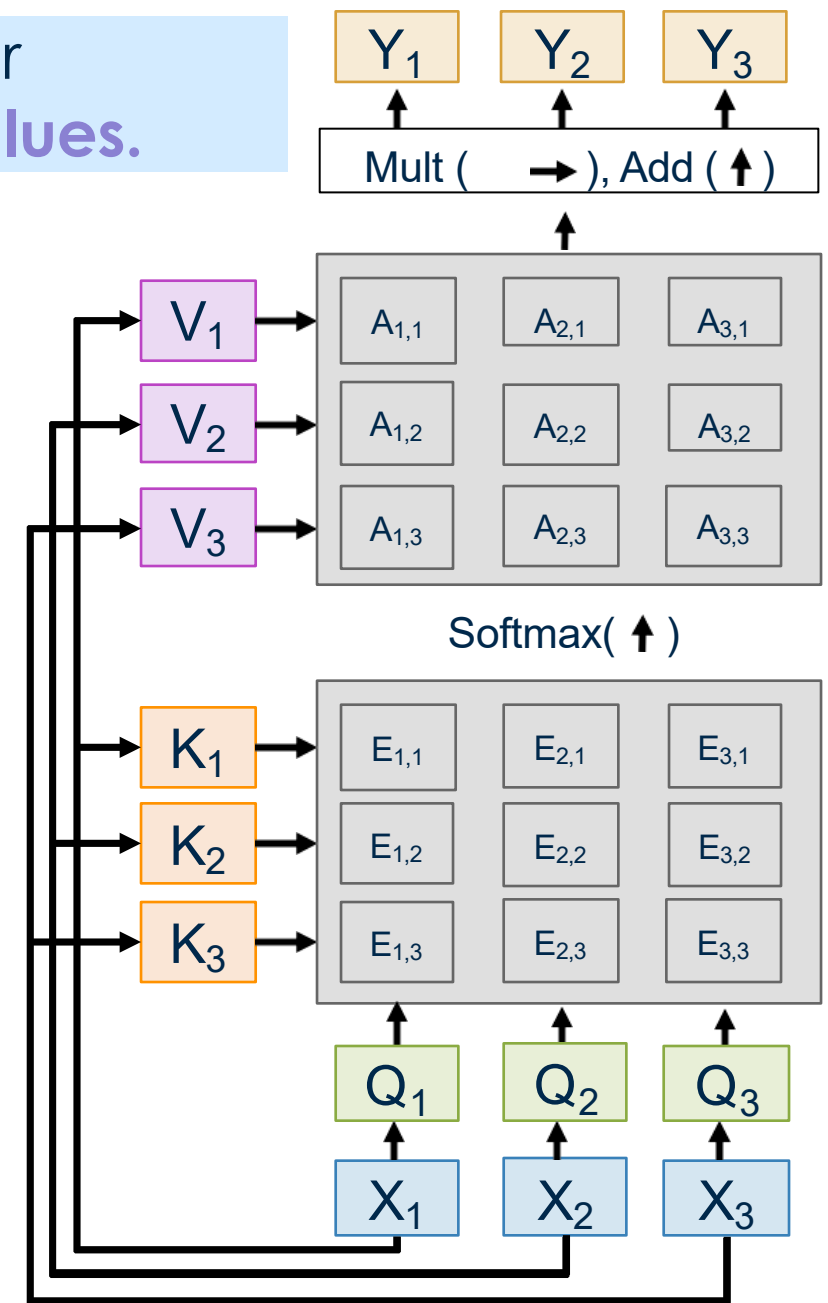
Queries: $\mathbf{Q} = \mathbf{XW}_Q$ [$N \times d_{out}$]

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{d_k}$ [$N \times N$]
 $E_{ij} = \mathbf{Q}_i \cdot \mathbf{K}_j / \sqrt{d_k}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$

Output vector: $\mathbf{Y} = \mathbf{AV}$ [$N \times d_{out}$]

$$Y_i = \sum_j A_{ij} V_j$$



Self-attention layer

Consider permuting **inputs**: **queries**, **keys**, and **values** will be the same but permuted

Similarities are the same but permuted

Attention weights are the same but permuted

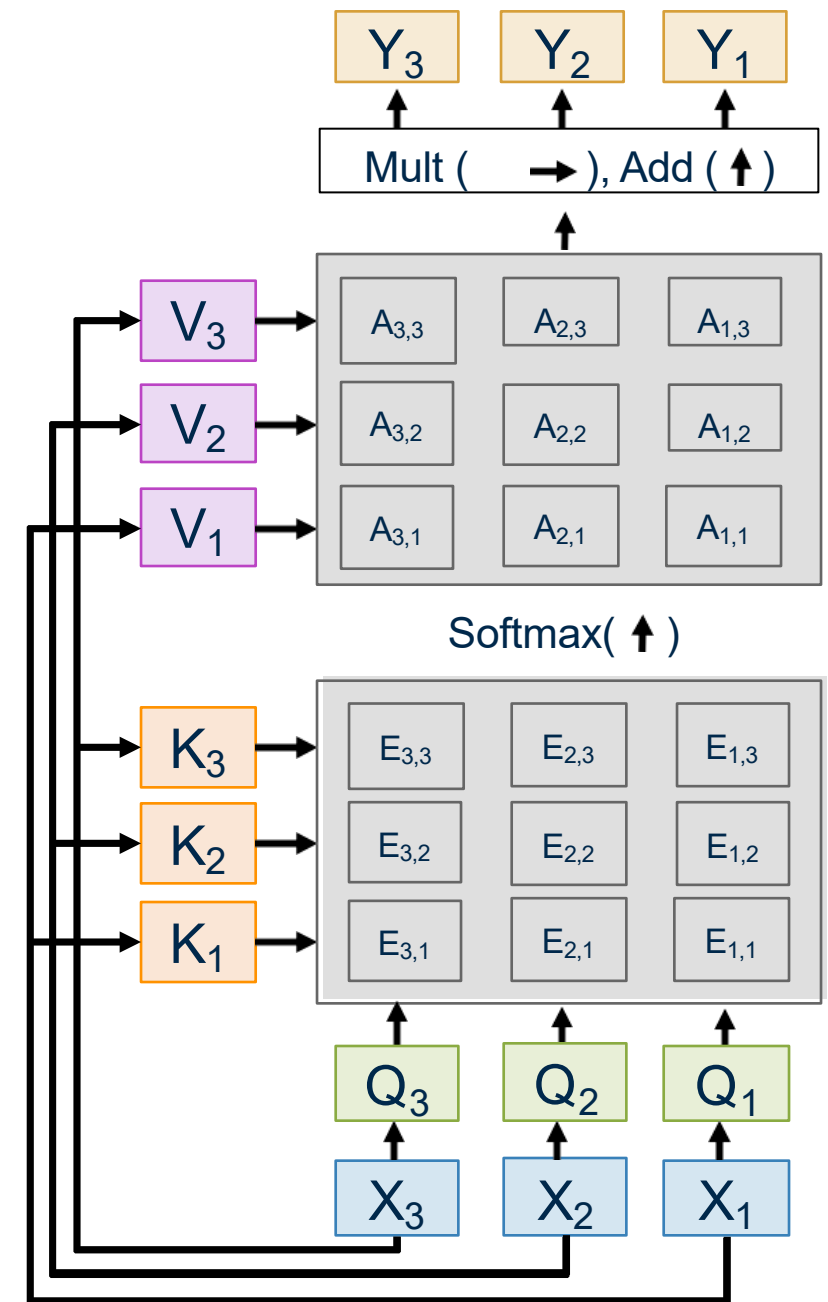
Outputs are the same but permuted

Self-Attention is **permutation equivariant**:

$$F(\sigma(X)) = \sigma(F(X))$$

→ self-attention works on **sets of vectors**

→ self-attention does not recognize the **order** of a sequence!



Self-attention layer

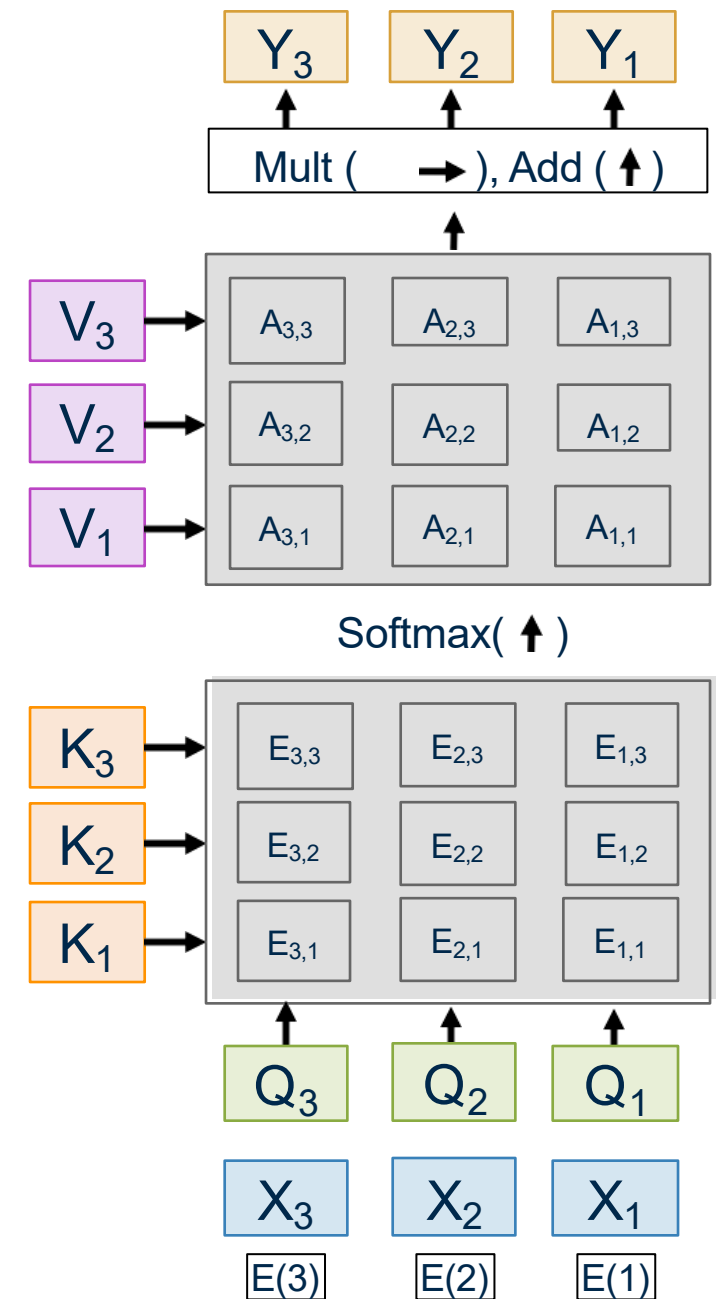
Self-Attention is **permutation equivariant**:

$$F(\sigma(X)) = \sigma(F(X))$$

→ self-attention works on **sets of vectors**

→ self-attention does not recognize the **order** of a sequence!

Solution: Add **positional encoding** to each input;
this is a vector that is a fixed function of the index



Masked attention layer

Don't let vectors “**look ahead**” in the sequence:

- Override similarities with $-\infty$; this controls which inputs each vector is allowed to look at.
- Useful for language model when you want to predict the next word

Computation:

Keys: $\mathbf{K} = \mathbf{XW_K} \quad [N \times d_{out}]$

Values: $\mathbf{V} = \mathbf{XW_V} \quad [N \times d_{out}]$

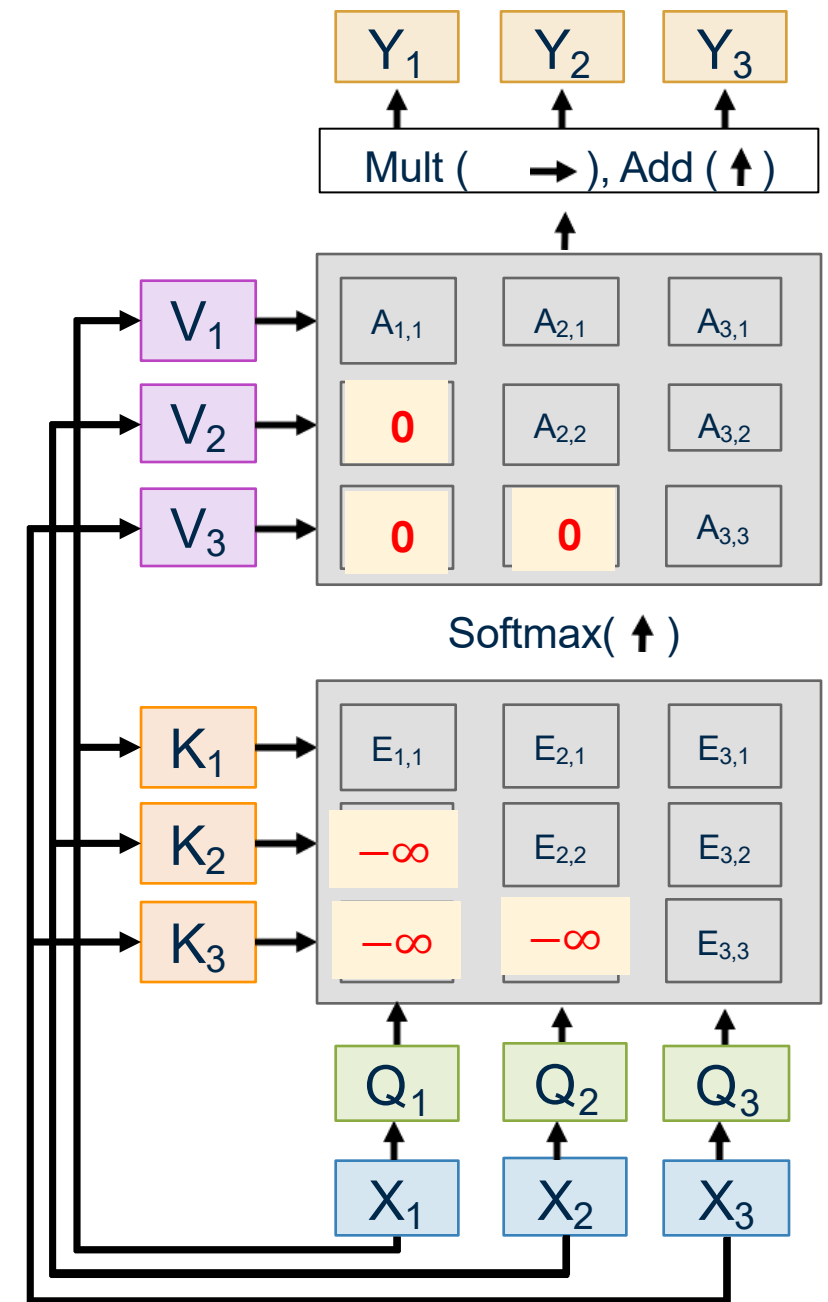
Queries: $\mathbf{Q} = \mathbf{XW_Q} \quad [N \times d_{out}]$

Similarities: $\mathbf{E} = \mathbf{QK^T} / \sqrt{d_k} \quad [N \times N]$
 $E_{ij} = \mathbf{Q_i} \cdot \mathbf{K_j} / \sqrt{d_k}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$

Output vector: $\mathbf{Y} = \mathbf{AV} \quad [N \times d_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$



Masked attention layer

Don't let vectors “**look ahead**” in the sequence:

- Override similarities with $-\infty$; this controls which inputs each vector is allowed to look at.
- Useful for language model when you want to predict the next word

Computation:

Keys: $\mathbf{K} = \mathbf{XW_K} \quad [N \times d_{\text{out}}]$

Values: $\mathbf{V} = \mathbf{XW_V} \quad [N \times d_{\text{out}}]$

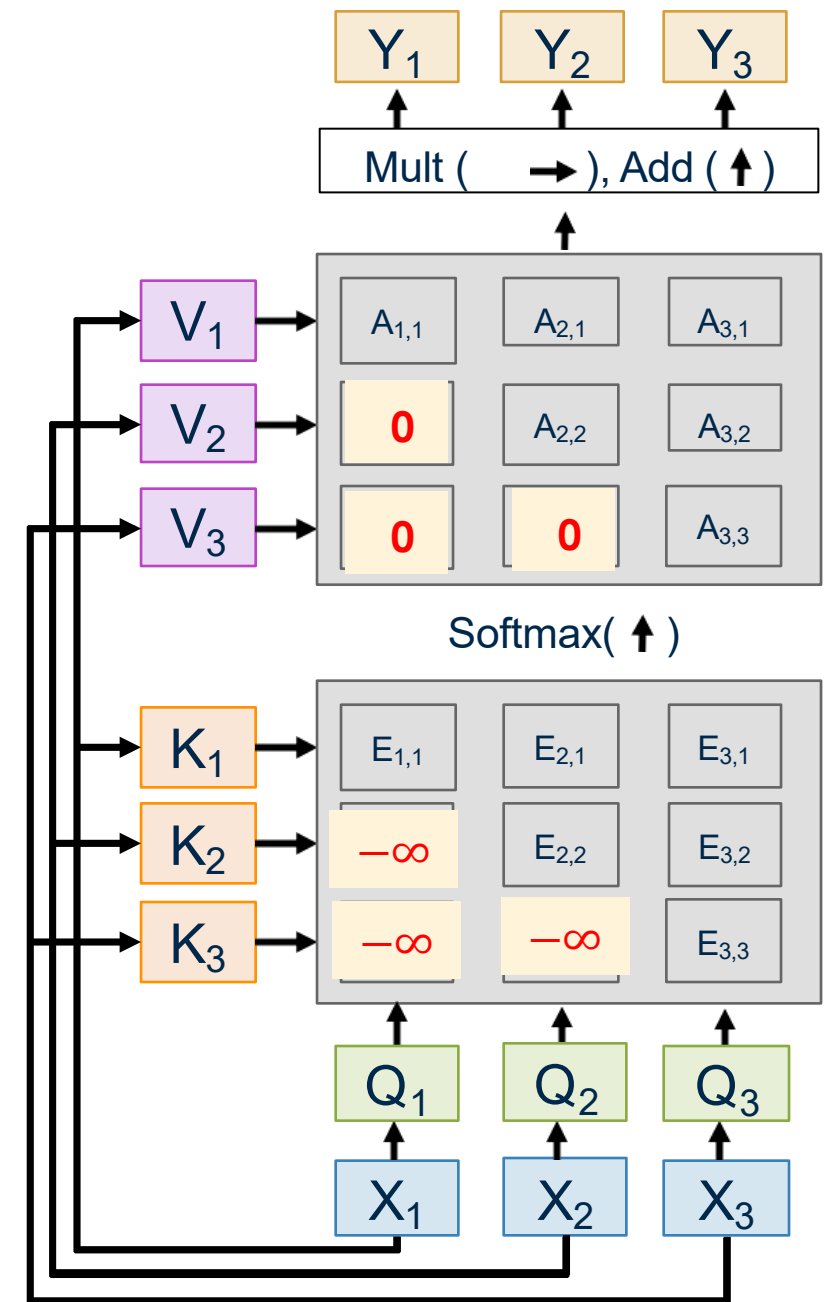
Queries: $\mathbf{Q} = \mathbf{XW_Q} \quad [N \times d_{\text{out}}]$

Similarities: $\mathbf{E} = \mathbf{QK^T} / \sqrt{d_k} \quad [N \times N]$
 $E_{ij} = \mathbf{Q_i} \cdot \mathbf{K_j} / \sqrt{d_k}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$

Output vector: $\mathbf{Y} = \mathbf{AV} \quad [N \times d_{\text{out}}]$

$$Y_i = \sum_j A_{ij} V_j$$



Masked attention layer

Don't let vectors “**look ahead**” in the sequence:

- Override similarities with $-\infty$; this controls which inputs each vector is allowed to look at.
- Useful for language model when you want to predict the next word

Computation:

Keys: $\mathbf{K} = \mathbf{XW_K}$ $[N \times d_{out}]$

Values: $\mathbf{V} = \mathbf{XW_V}$ $[N \times d_{out}]$

Queries: $\mathbf{Q} = \mathbf{XW_Q}$ $[N \times d_{out}]$

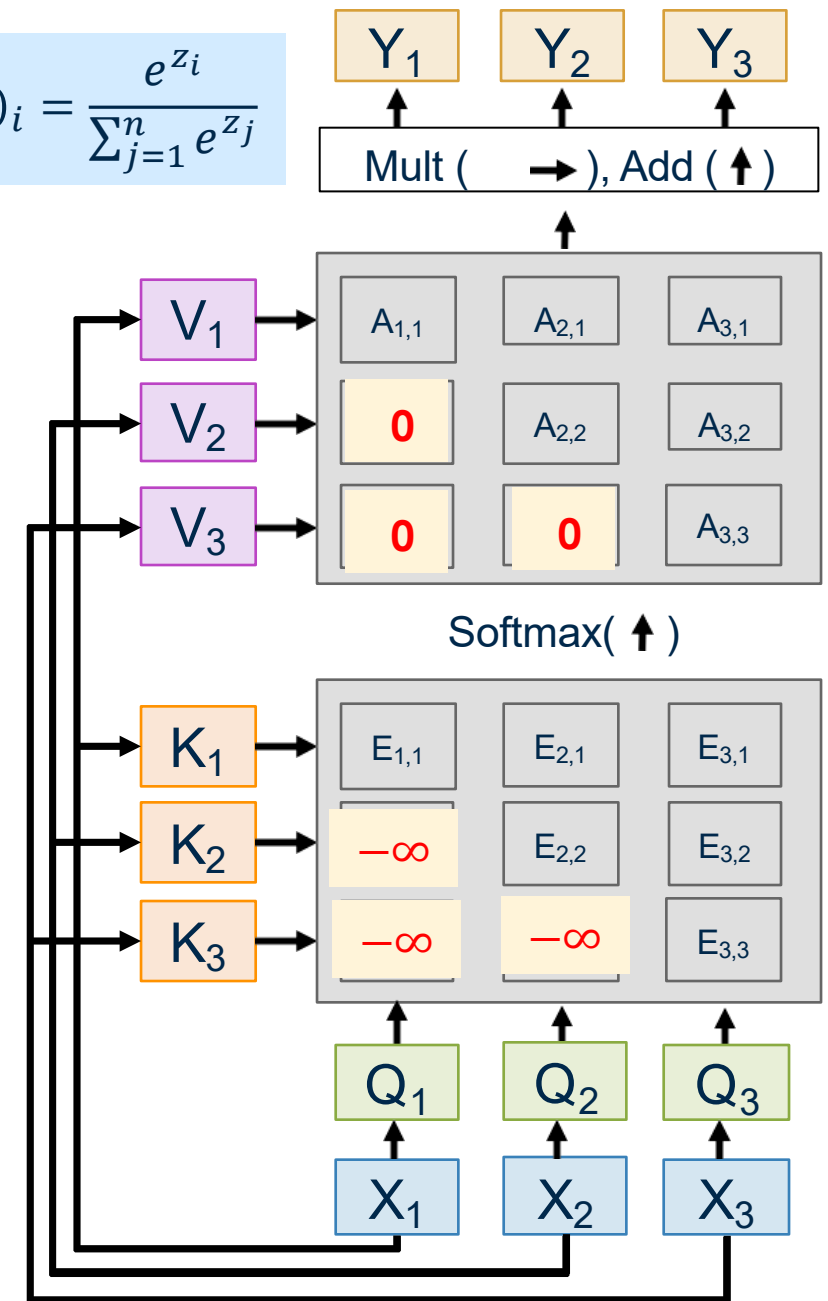
Similarities: $\mathbf{E} = \mathbf{QK^T} / \sqrt{d_k}$ $[N \times N]$
 $E_{ij} = \mathbf{Q_i} \cdot \mathbf{K_j} / \sqrt{d_k}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$

Output vector: $\mathbf{Y} = \mathbf{AV}$ $[N \times d_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$



Masked attention layer

In practice: we define an attention mask:

$$M_{ij} = \begin{cases} 0 & \text{if visible token} \\ -\infty & \text{if future token} \end{cases}$$

and add it to the similarity values

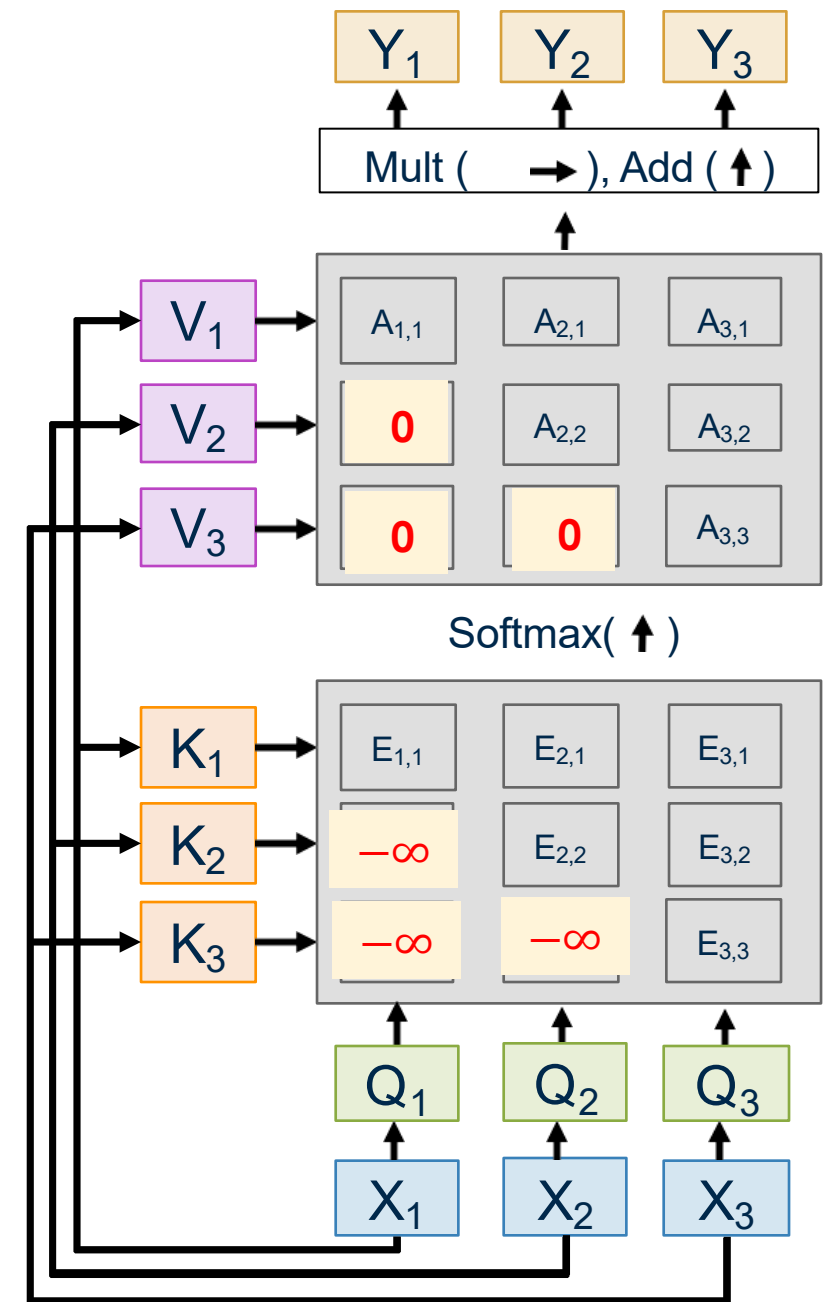
$$\widetilde{e}_{ij} = \frac{\mathbf{q}_i \cdot \mathbf{k}_j}{\sqrt{d_k}} + M_{ij}$$

Considering the definition of softmax

$$\text{softmax}(\mathbf{z})_i = \frac{e^{z_i}}{\sum_{j=1}^n e^{z_j}}$$

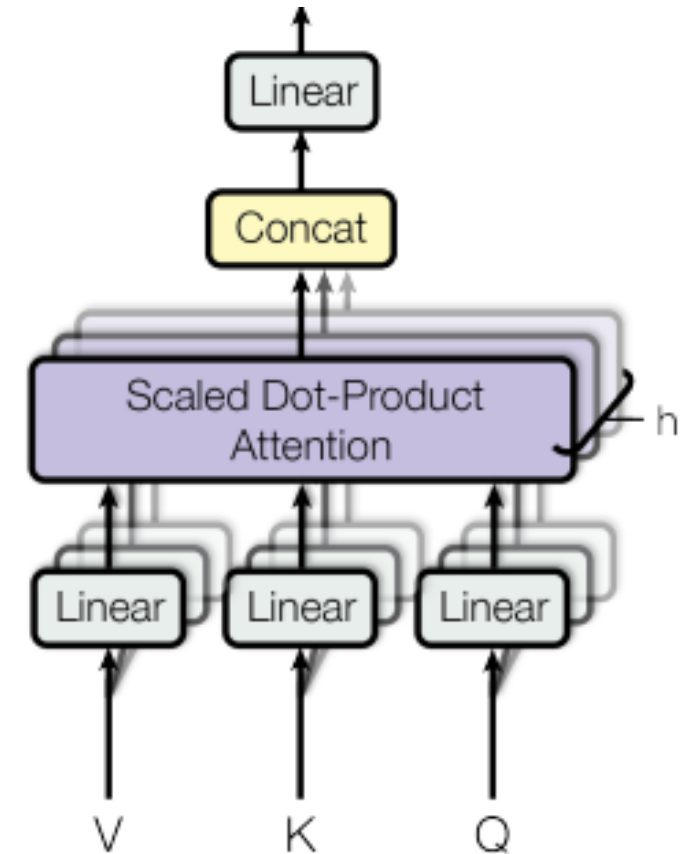
and recalling $\lim_{x \rightarrow -\infty} e^x = 0$

$$\text{softmax}(\mathbf{z})_i = 0 \text{ if } z_i = -\infty$$



Multi-head attention

- Multi-Head Attention consists of several attention layers running in parallel
- All weights are replicated H times, where H is the number of heads



Self-Attention as matrix multiplications

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[N \times d_{out}]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[N \times d_{out}]$

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[N \times d_{out}]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{d_k}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$

Output vector: $\mathbf{Y} = \mathbf{AV}$ $[N \times d_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$

Recall:

- N is the number of inputs
- H is the number of heads

Question: how much memory does it take to store attention weights?

Self-Attention as matrix multiplications

Computation:

Keys: $\mathbf{K} = \mathbf{XW}_K$ $[N \times d_{out}]$

Values: $\mathbf{V} = \mathbf{XW}_V$ $[N \times d_{out}]$

Queries: $\mathbf{Q} = \mathbf{XW}_Q$ $[N \times d_{out}]$

Similarities: $\mathbf{E} = \mathbf{QK}^T / \sqrt{d_k}$

Attention weights: $\mathbf{A} = \text{softmax}(\mathbf{E}, \text{dim}=1)$

Output vector: $\mathbf{Y} = \mathbf{AV}$ $[N \times d_{out}]$

$$Y_i = \sum_j A_{ij} V_j$$

Recall:

- N is the number of inputs
- H is the number of heads

Question: how much memory does it take to store attention weights?

If $N=100K$, $H=64$ then $H \times N \times N$ attention weights take **1.192 TB!**

Solution: Flash Attention allows to compute the attention matrix in blocks, streaming computation of the softmax function, without the need to materialize the full attention matrix

<https://www.youtube.com/watch?v=eJn6zsbrqUM&t=140>

The Transformer

Transformer Block

Input: Set of vectors x

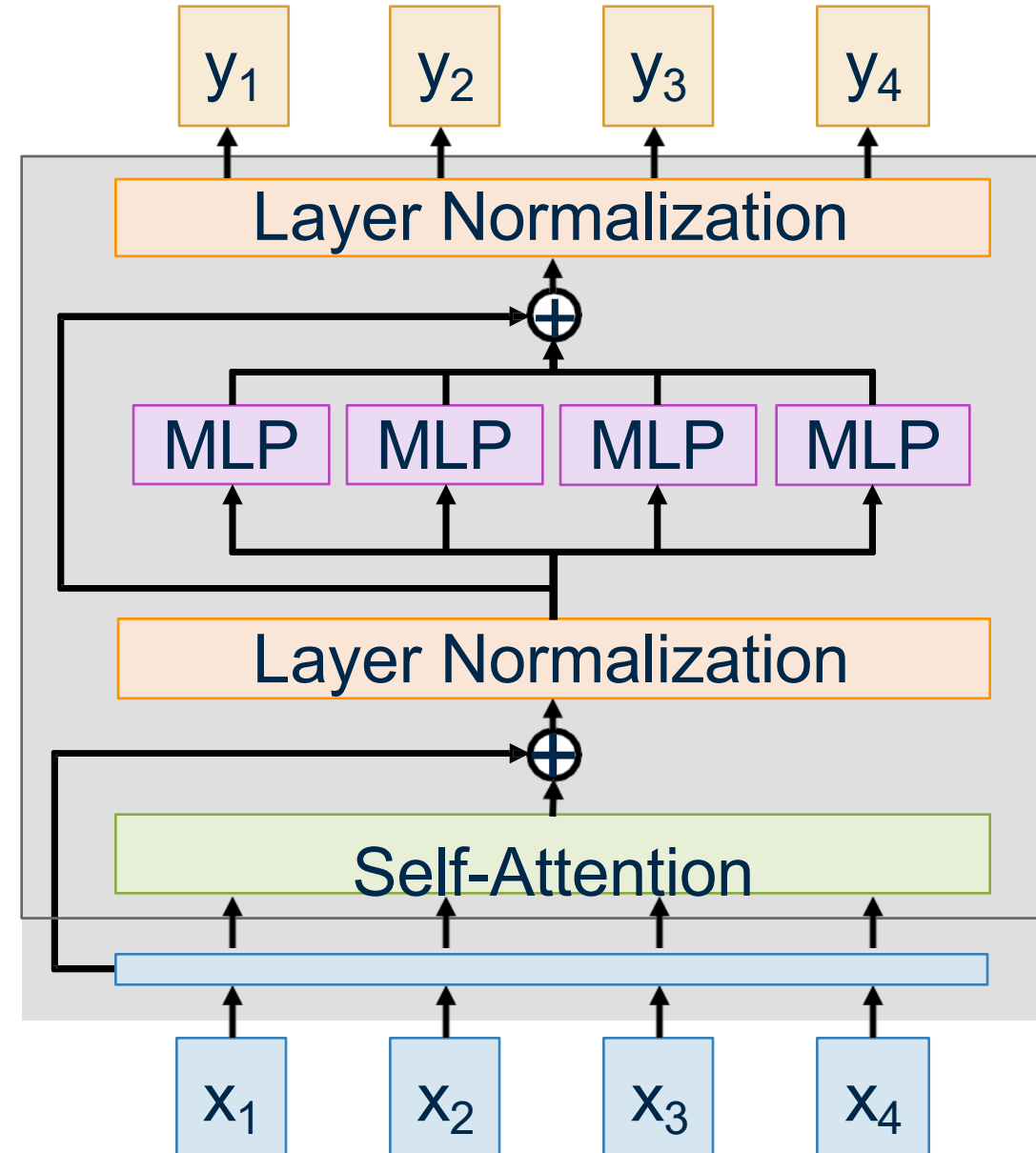
Output: Set of vectors y

Self-Attention is the only interaction between vectors

LayerNorm and MLP work on each vector independently

Highly scalable and parallelizable, most of the compute is just 6 matmuls:

4 from Self-Attention
2 from MLP



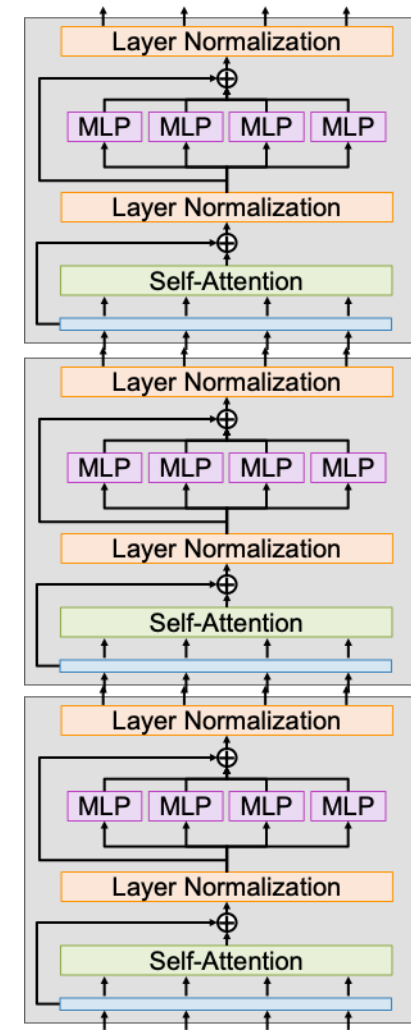
The Transformer

A **Transformer** is just a stack of identical Transformer blocks!

Original: [Vaswani et al, 2017]
12 blocks, D=1024, H=16, N=512
213M params

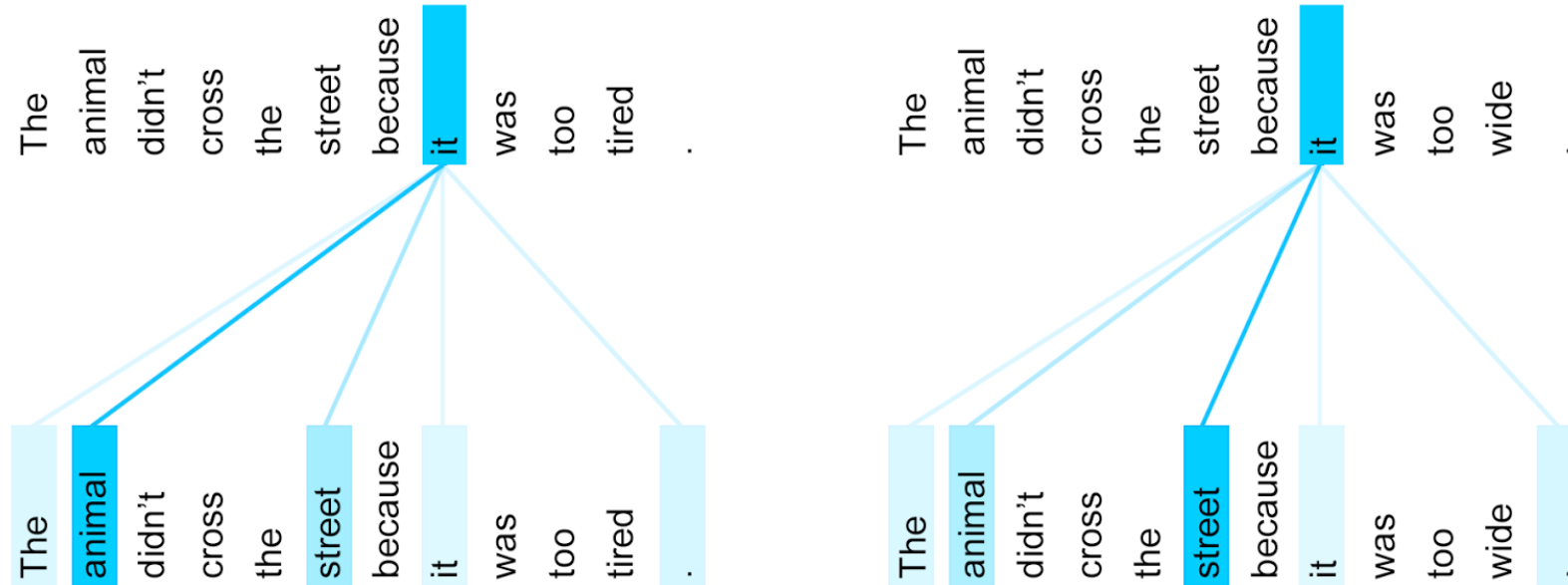
GPT-2: [Radford et al, 2019]
48 blocks, D=1600, H=25, N=1024
1.5B params

GPT-3: [Brown et al, 2020]
96 blocks, D=12288, H=96, N=2048
175B params



Self-attention at play

Self-attention layers learnt “it” could refer to different entities in different contexts

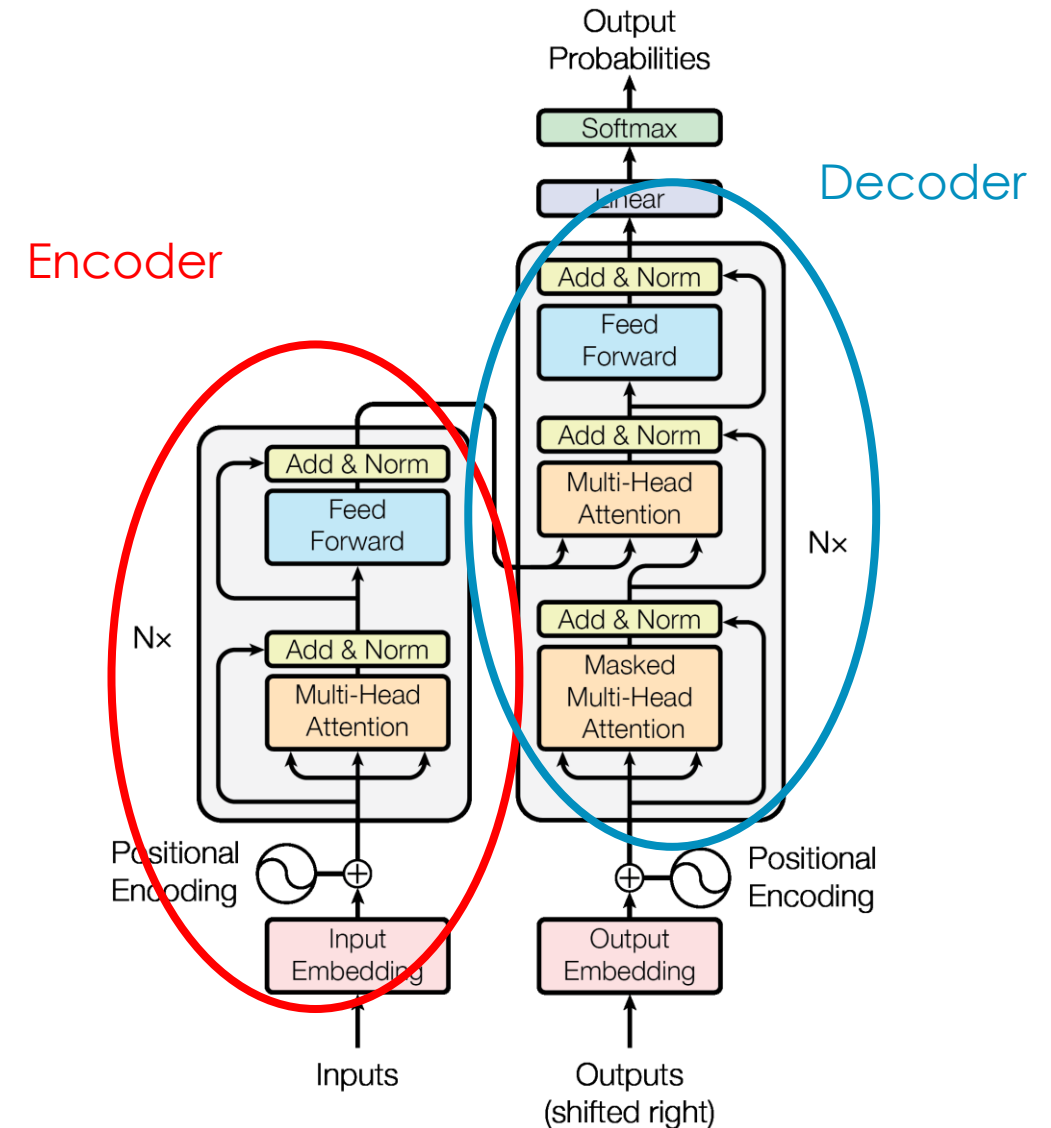


<https://ai.googleblog.com/2017/08/transformer-novel-neural-network.html>

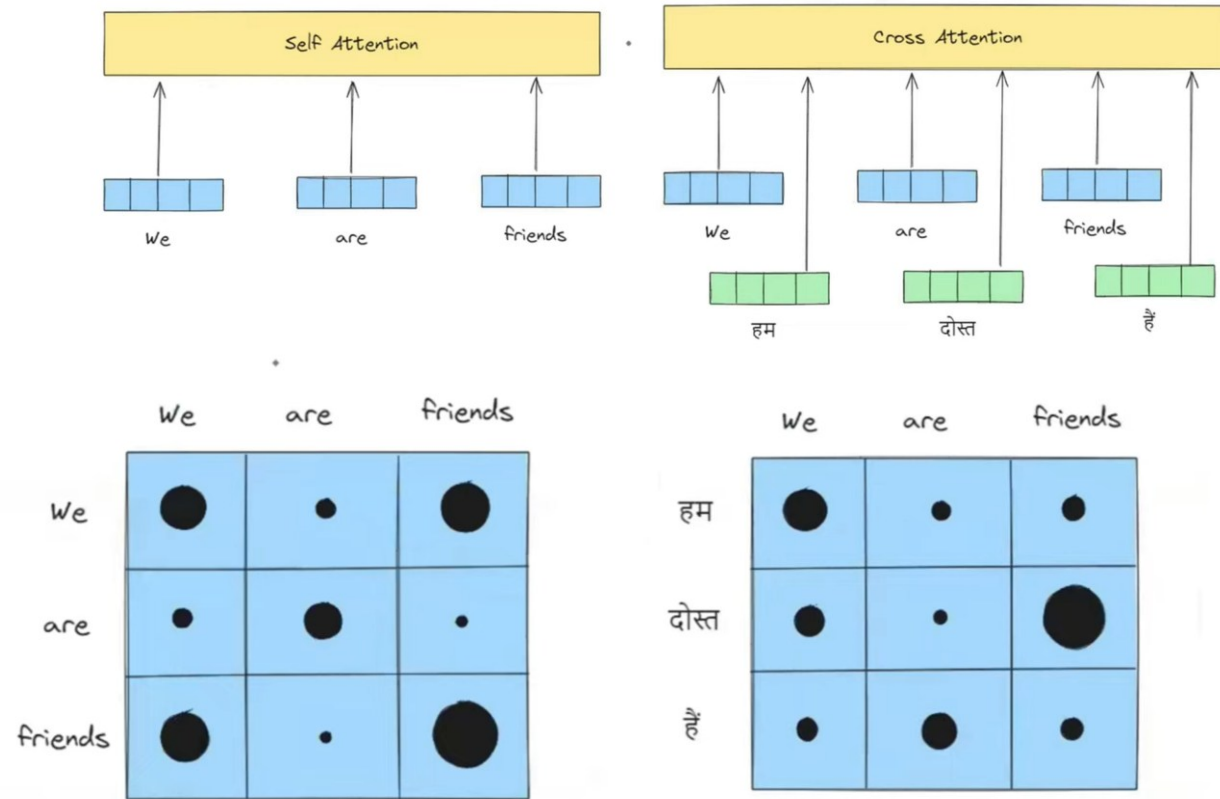
Transformer architecture – T5

- Encoder builds a rich context representation; decoder attends to it via cross-attention.
- Use decoder-only architectures for language generation
- Encoder-decoder architectures for text-to-text tasks
 - language translation
 - language summarization

[Vaswani et. al., 2017. Attention is all you need]

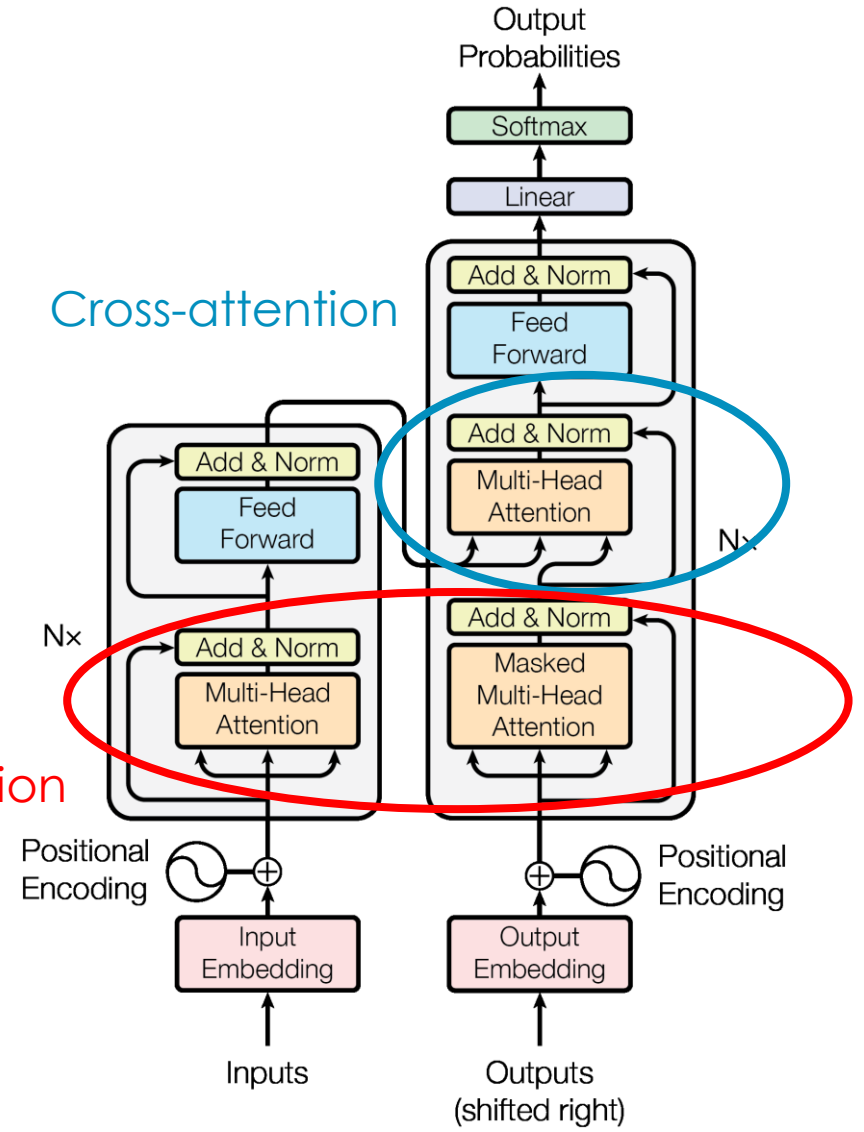


Transformer architecture – T5



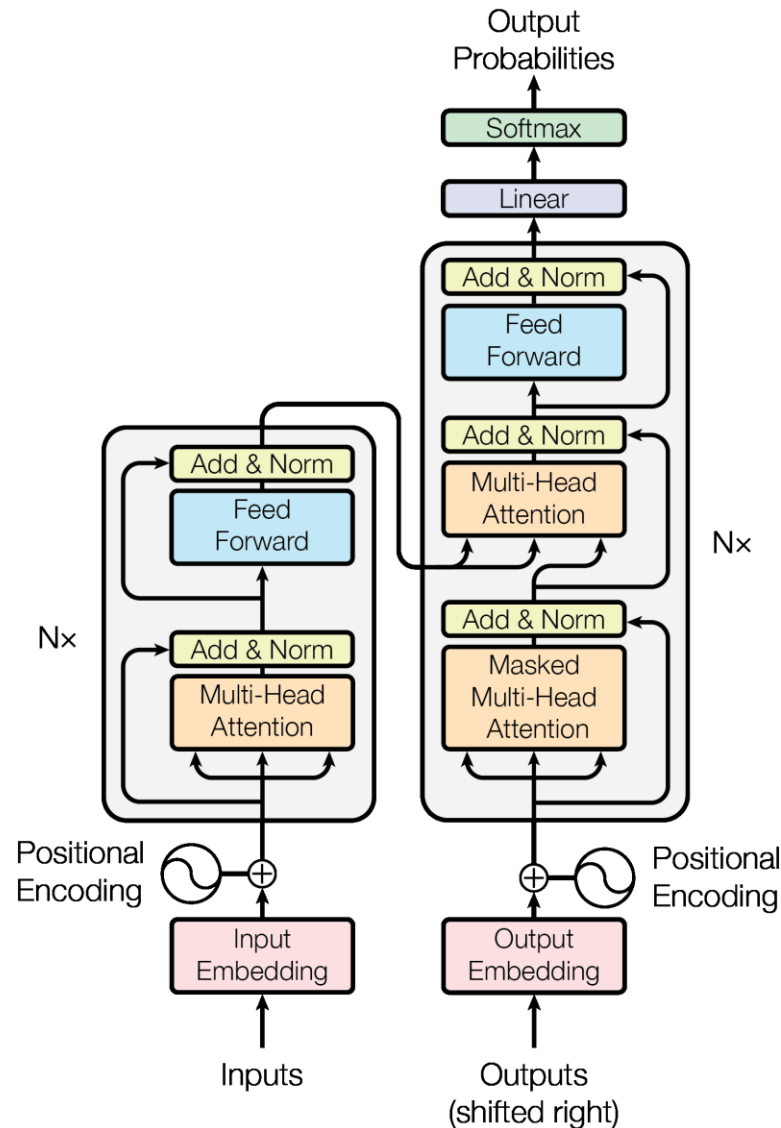
all you need

<https://medium.com/@sachinsoni600517/cross-attention-in-transformer-f37ce7129d78>



Position embedding

Problem: the output of the attention layer does not depend on the order!

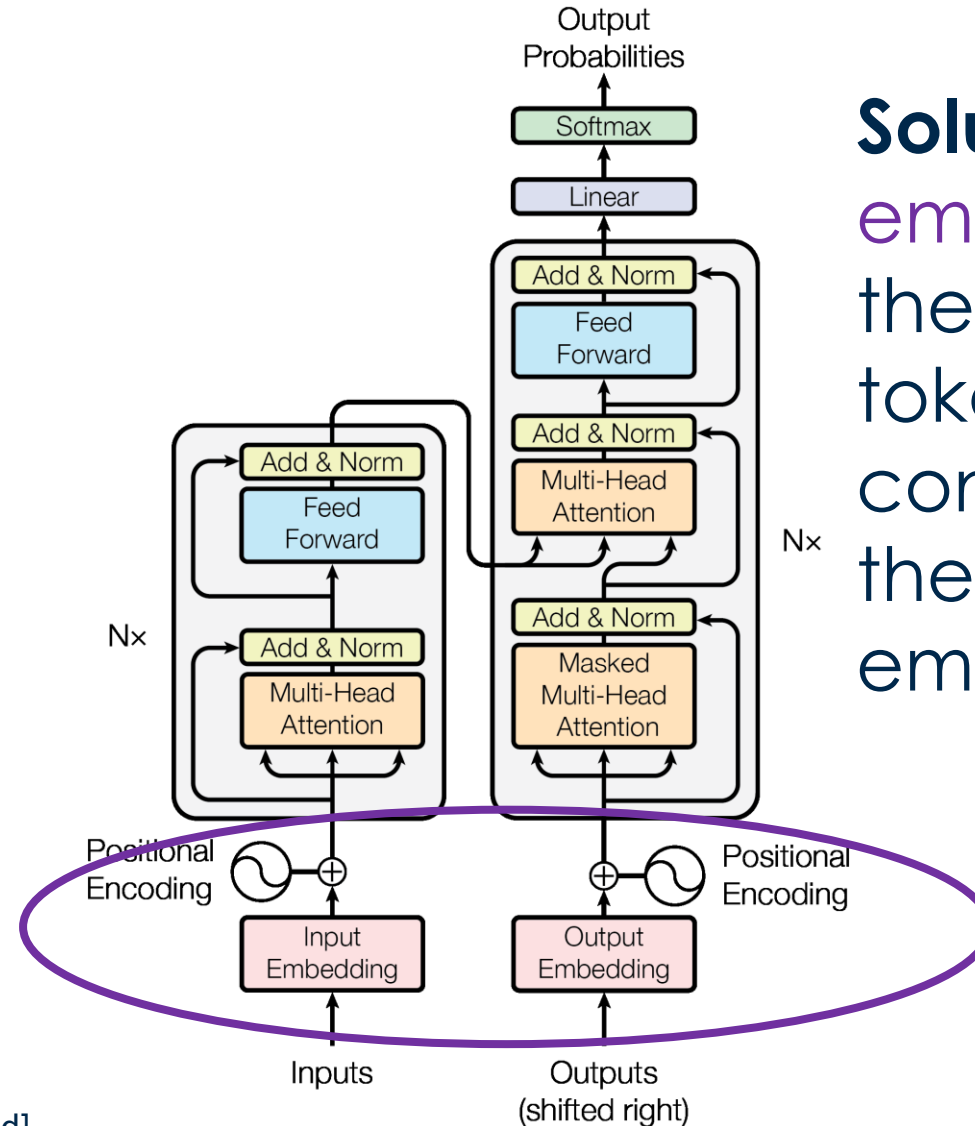


[Vaswani et. al., 2017. Attention is all you need]

Position embedding

Problem: the output of the attention layer does not depend on the order!

Solution: positional embedding encode the position of each token and are concatenated to the input embedding.



[Vaswani et. al., 2017. Attention is all you need]



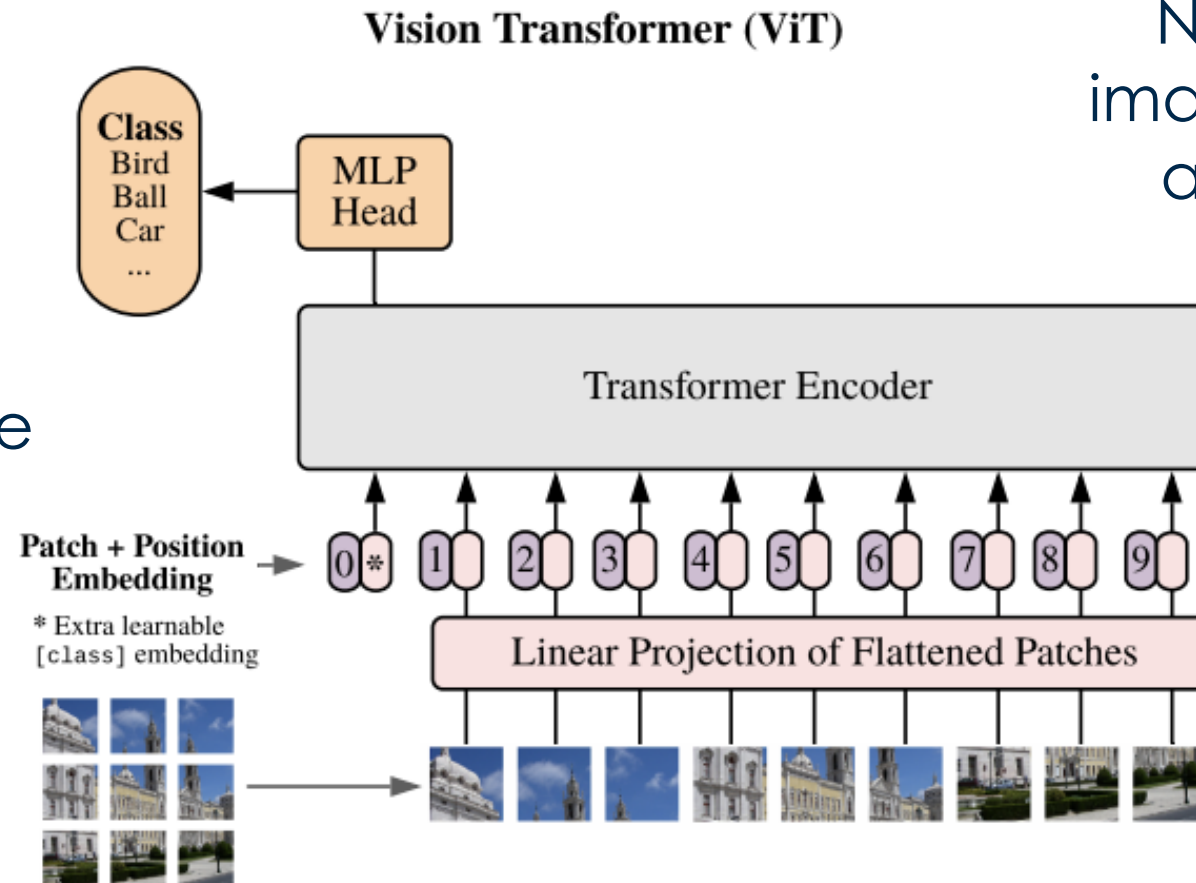
Politecnico
di Torino

From text to images —
the same machinery



Images as sequences: ViT

Use positional encoding to tell the transformer the 2D position of each patch



No masking; each image patch can look at all other image patches

[Dosovitskiy et al., “An image is worth 16x16 words”, ICLR, 2021.]

What do ViT tell us?

- At **sufficient scale** (large datasets + large models), ViT matches or surpasses convolutional networks on vision benchmarks
 - CNNs bake in inductive biases: translation equivariance, locality
 - Inductive biases are not strictly necessary; they can be learned from data if the dataset is large enough
 - Shifts the design philosophy towards scalable, data-hungry general models
 - Set the stage for unified architectures across modalities (text, image, audio)

CLIP - motivation

- Traditional computer vision systems are trained with a fixed set of predetermined object categories.
- Can we learn a better representation from natural language supervision?



"man in black shirt is playing guitar."



"construction worker in orange safety vest is working on road."



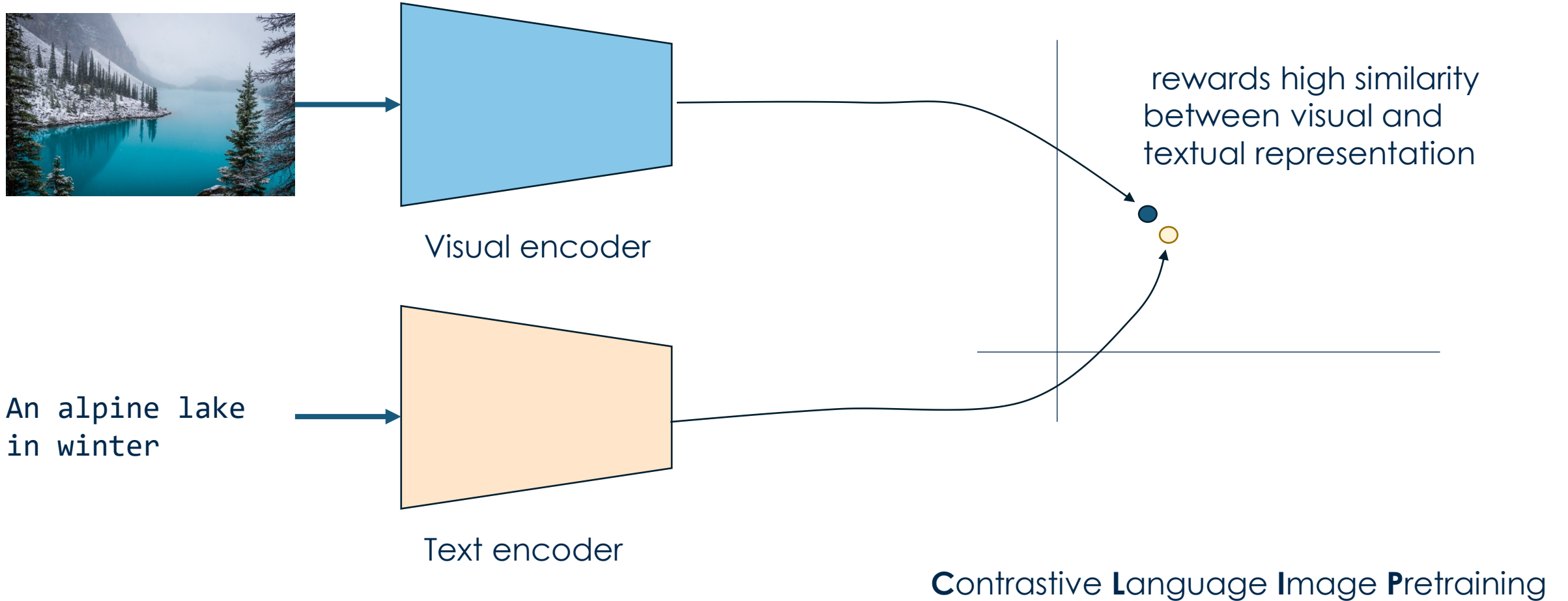
"girl in pink dress is jumping in air."



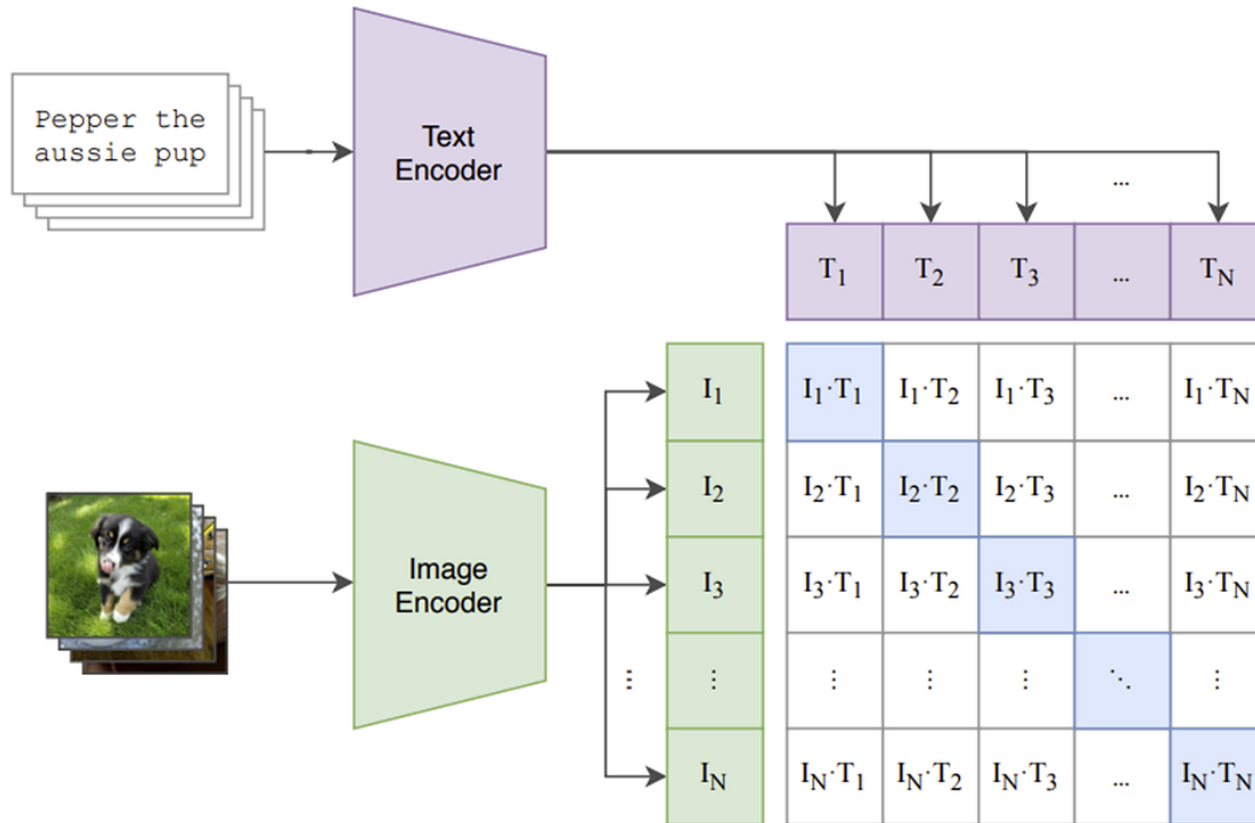
"black and white dog jumps over bar."

Alec Radford, "Learning Transferable Visual Models From Natural Language Supervision", ICML 2021

Training objective (high-level)



CLIP - architecture



- For each batch, maximize cosine similarity between matched (image, caption) pairs; minimize it for all other pairings
- Loss function: InfoNCE / contrastive loss
- Intuitively, each image v_i must **identify its own caption t_i out of N distractors**, and vice-versa

Alec Radford, "Learning Transferable Visual Models From Natural Language Supervision", ICML 2021

CLIP - loss

$$\mathcal{L}_{CLIP} = \frac{1}{2} (\mathcal{L}_{img \rightarrow txt} + \mathcal{L}_{txt \rightarrow img})$$

$$\mathcal{L}_{img \rightarrow txt} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s_{ii})}{\sum_{j=1}^N \exp(s_{ij})}$$

$$s_{ij} = \frac{f_V(v_i)^\top f_T(t_j)}{|f_V(v_i)| \cdot |f_T(t_j)|} \cdot \exp(\tau)$$

$$\sum_{j=1}^N \exp(s_{ij})$$

$$-\log(\cdot)$$

$$\frac{1}{N} \sum_i$$

Symmetry ($\frac{1}{2}$ avg)

$$\mathcal{L}_{txt \rightarrow img} = -\frac{1}{N} \sum_{i=1}^N \log \frac{\exp(s_{ii})}{\sum_{j=1}^N \exp(s_{ji})}$$

Numerator — rewards high similarity for the correct pair (v_i, t_i) . Temperature controls how sharp the distribution is

Denominator — sums over all N texts in the batch, including $N - 1$ negatives

Converts a probability into a loss: perfect match
→ probability 1 → loss 0

Averages over all anchors in the batch

Ensures both encoders receive gradients;
neither encoder is treated as "fixed"

Training loop - sketch

```
# image_encoder - ResNet or Vision Transformer
# text_encoder  - CBOW or Text Transformer
# I[n, h, w, c] - minibatch of aligned images
# T[n, l]       - minibatch of aligned texts
# W_i[d_i, d_e] - learned proj of image to embed
# W_t[d_t, d_e] - learned proj of text to embed
# t            - learned temperature parameter

# extract feature representations of each modality
I_f = image_encoder(I) #[n, d_i]
T_f = text_encoder(T)  #[n, d_t]

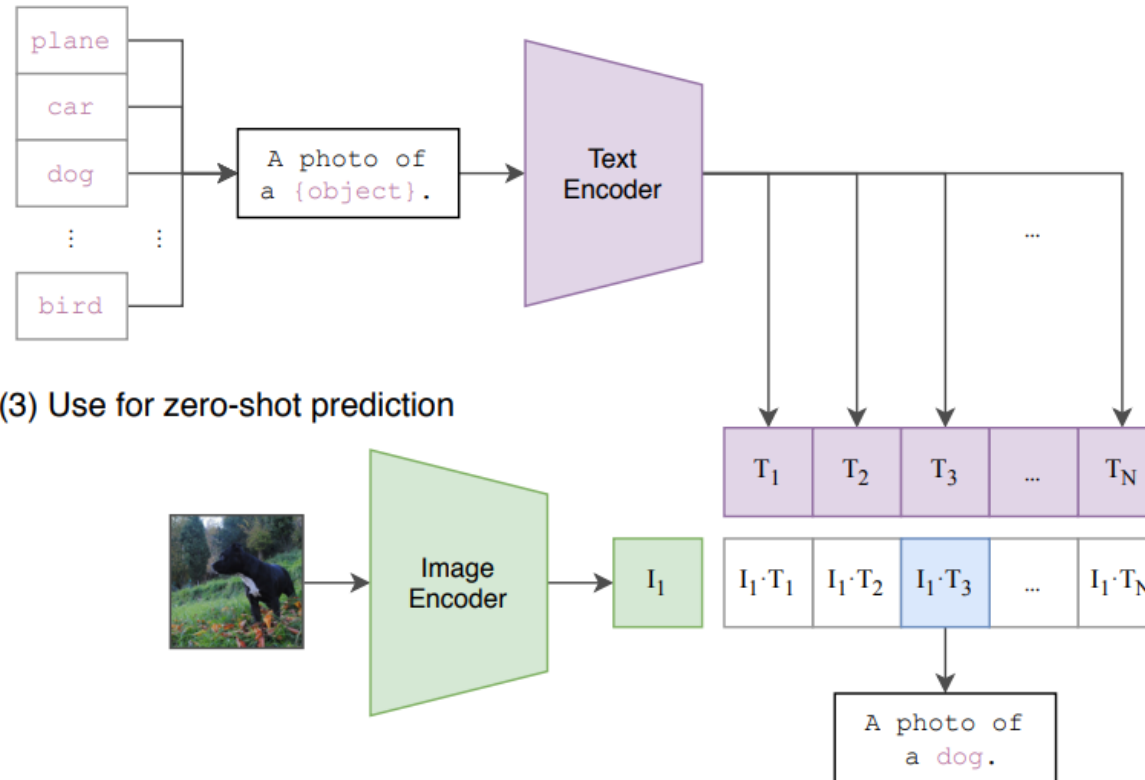
# joint multimodal embedding [n, d_e]
I_e = l2_normalize(np.dot(I_f, W_i), axis=1)
T_e = l2_normalize(np.dot(T_f, W_t), axis=1)

# scaled pairwise cosine similarities [n, n]
logits = np.dot(I_e, T_e.T) * np.exp(t)

# symmetric loss function
labels = np.arange(n)
loss_i = cross_entropy_loss(logits, labels, axis=0)
loss_t = cross_entropy_loss(logits, labels, axis=1)
loss   = (loss_i + loss_t)/2
```

CLIP for zero-shot classification

(2) Create dataset classifier from label text



Alec Radford, "Learning Transferable Visual Models From Natural Language Supervision", ICML 2021

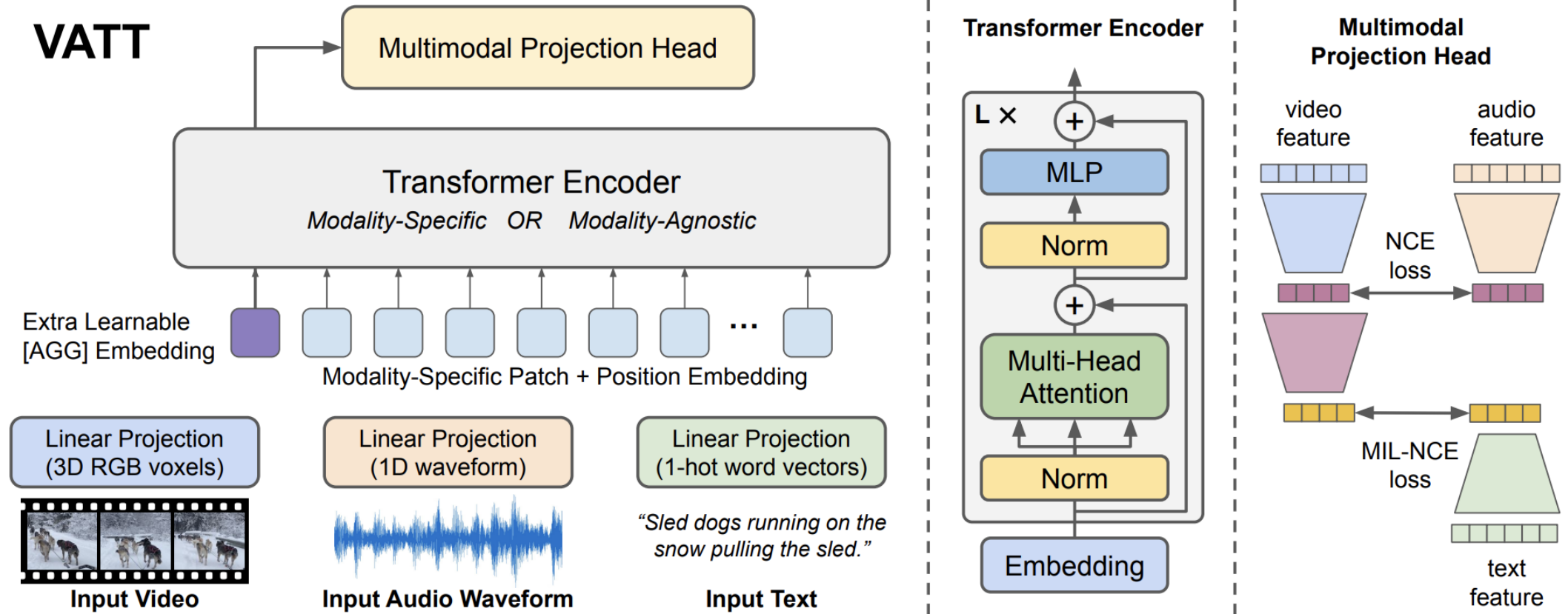
Why CLIP is fundamental for visual genAI?

- CLIP's text encoder is the conditioning backbone inside Stable Diffusion — it converts prompts into the vectors that guide image generation
- Compare an image embedding against text embeddings of class labels (e.g. "a photo of a cat") — no fine-tuning needed
 - Powers zero-shot classification without any task-specific training data
 - Enables image-text retrieval at scale
- CLIP score (cosine similarity between generated image and text prompt) is a standard automatic evaluation metric for text-to-image models
- Nearly every topic in this course — diffusion models, latent diffusion, RL from human feedback — touches CLIP in some way

Variants and limitations

- Limitation of CLIP: 77-token hard limit on caption.
 - Comes from the fixed positional embedding table in the original Transformer text encoder (inherited from GPT-2)
 - Long or complex prompts in Stable Diffusion are silently truncated at token words beyond that threshold have zero influence on generation
- Several variants extend the original CLIP: OpenCLIP (open-source reproduction), ALIGN (larger noisier data), SigLIP (sigmoid loss instead of softmax, more stable at large batch sizes), FLAVA (adds multimodal fusion head)
- Workarounds: prompt weighting, prompt chunking (used in ComfyUI), or switching to architectures with longer context (e.g. SD 3 uses T5-XXL with up to 256 tokens)

Multi-modal transformers



[Akbari, VATT: Transformers for Multimodal Self-Supervised Learning from Raw Video, Audio and Text, 2021]



Politecnico
di Torino

Autoregressive generation



Auto-regressive generation

The goal of any generative model is to learn the **joint distribution** over data:

$$p(\mathbf{x}) = p(x_1, x_2, \dots, x_D)$$

where $\mathbf{x} = (x_1, x_2, \dots, x_D)$ is a D -dimensional data point (e.g. a sequence of tokens, pixels, etc.)

In autoregressive models, we apply the **chain rule of probability** to obtain an exact, lossless decomposition of conditional probabilities.

Example



Let us say $D=5$. How do we rewrite

$$p(\mathbf{x}) = p(x_1, x_2, \dots, x_5)$$

as a product of conditional probabilities $p(x_i \mid x_1, \dots, x_{i-1})$?

Auto-regressive generation

The **chain rule of probability** is the exact, lossless decomposition:

$$p(x_1, x_2, \dots, x_D) = p(x_1) \cdot p(x_2 \mid x_1) \cdot p(x_3 \mid x_1, x_2) \cdots p(x_D \mid x_1, \dots, x_{D-1})$$

Written compactly with the notation $\mathbf{x}_{<\mathbf{i}} = (x_1, \dots, x_{i-1})$

$$p(\mathbf{x}) = \prod_{i=1}^D p(x_i \mid \mathbf{x}_{<\mathbf{i}})$$

This is an **exact identity** — no approximation is involved

Auto-regressive model

The recursion becomes explicit when we observe that each factor conditions on all **previously generated** elements. Define the partial joint up to position i :

$$P_i = p(x_1, \dots, x_i)$$

$$P_i = P_{i-1} \cdot p(x_i \mid \mathbf{x}_{<i}), \quad P_0 := 1$$

Then the chain rule gives the recurrence:

which "grows" the joint one variable at a time. Each step requires only the **previous context** $\mathbf{x}_{<i}$ making the computation inherently recursive.

An auto-regressive model **parameterises** each conditional factor with a shared neural network f_ϕ (e.g. a Transformer):

$$p_\phi(\mathbf{x}) = \prod_{i=1}^D p_\phi(x_i \mid \mathbf{x}_{<i})$$

Training an auto-regressive model

The log-likelihood decomposes into a **sum of independent terms**:

$$\log p_{\phi}(\mathbf{x}) = \sum_{i=1}^D \log p_{\phi}(x_i \mid \mathbf{x}_{<i})$$

- Train to predict next token using softmax + cross-entropy loss
- Teacher forcing during training (always feed previous ground truth)

Sampling an auto-regressive model

Generation proceeds by **sampling** — directly executing the recursive structure:

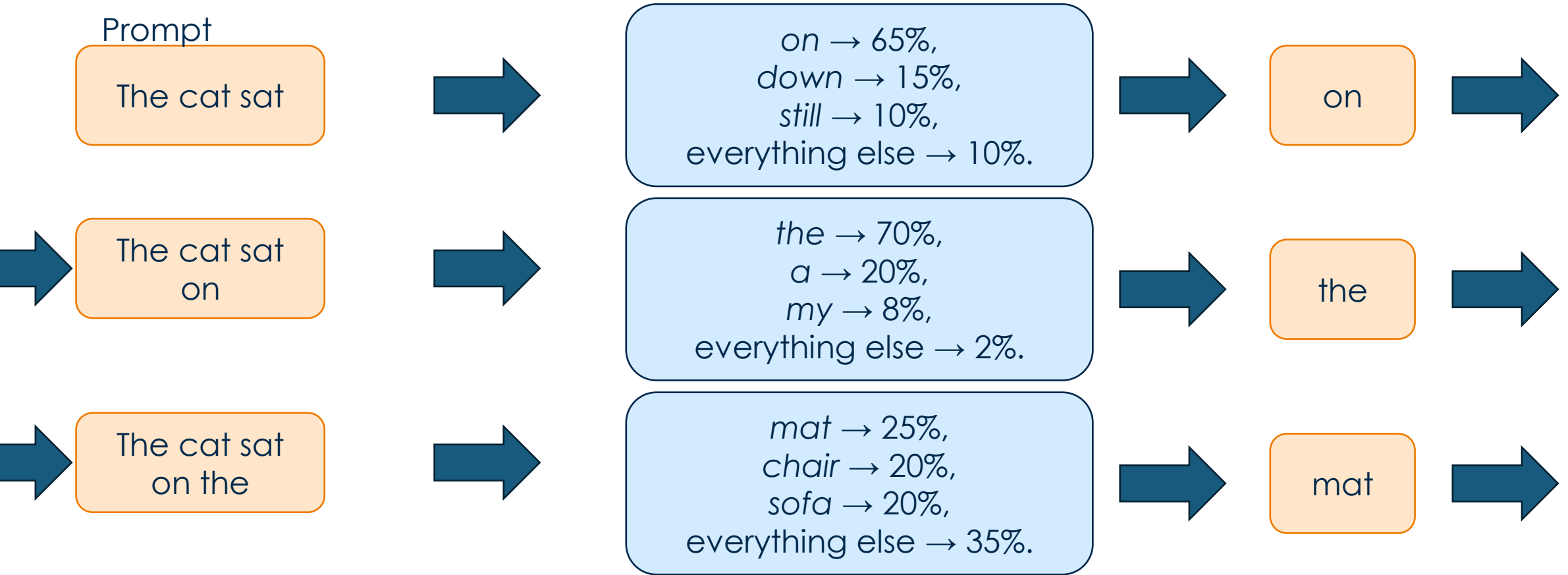
$$x_1 \sim p_\phi(x_1), \quad x_2 \sim p_\phi(x_2 \mid x_1), \quad \dots \quad x_D \sim p_\phi(x_D \mid \mathbf{x}_{<D})$$

Each sample depends **causally** on all prior ones. This is the recursive formulation **run forward in time**:

$$x_i \sim p_\phi(x_i \mid x_1, \dots, x_{i-1}), \quad i = 1, \dots, D$$

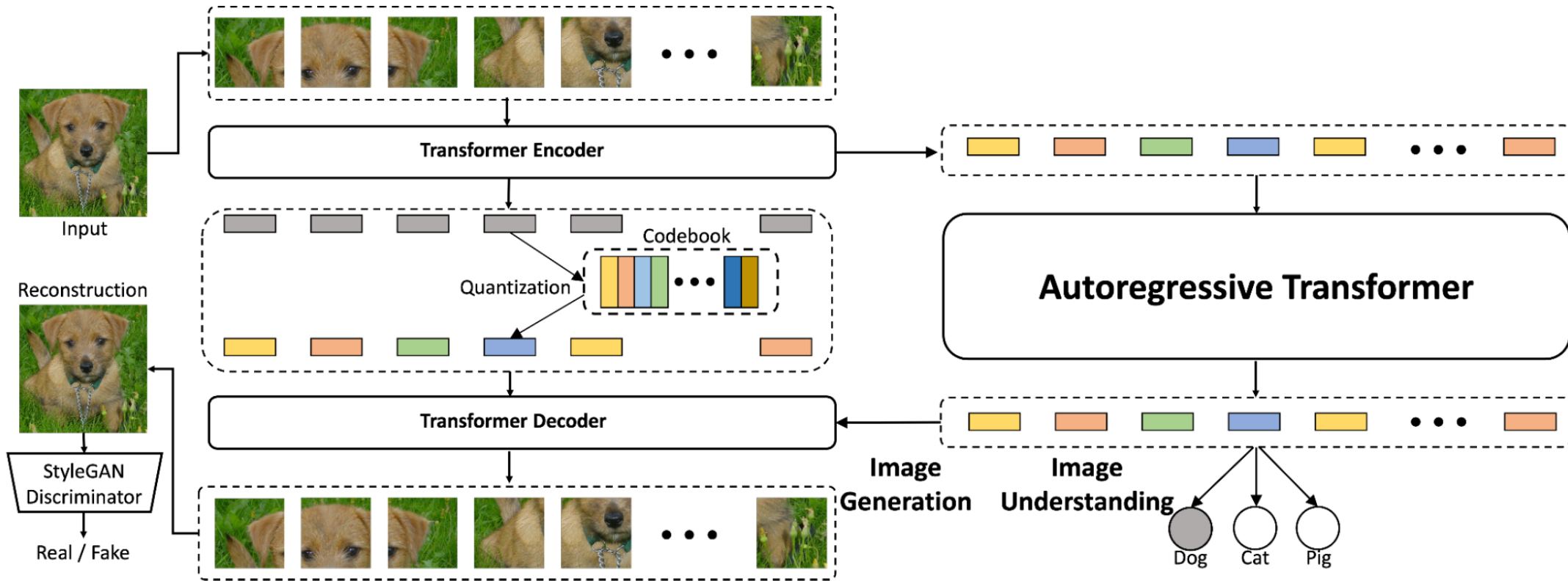
A concrete example – text generation

The model is asked, repeatedly: "given everything so far, what comes next?"
Probability distribution



until the end of the sentence is reached.

Images – patches as token

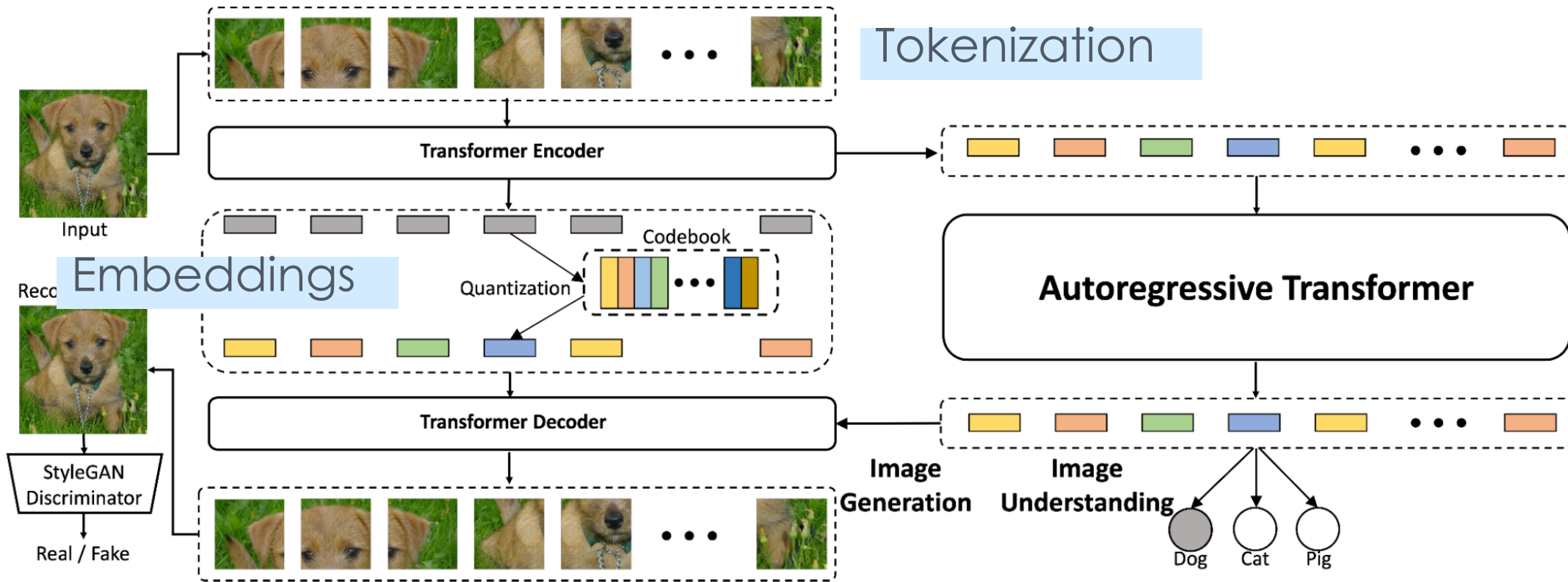


Stage 1: Image Quantization

Stage 2: Vector-quantized Image Modeling

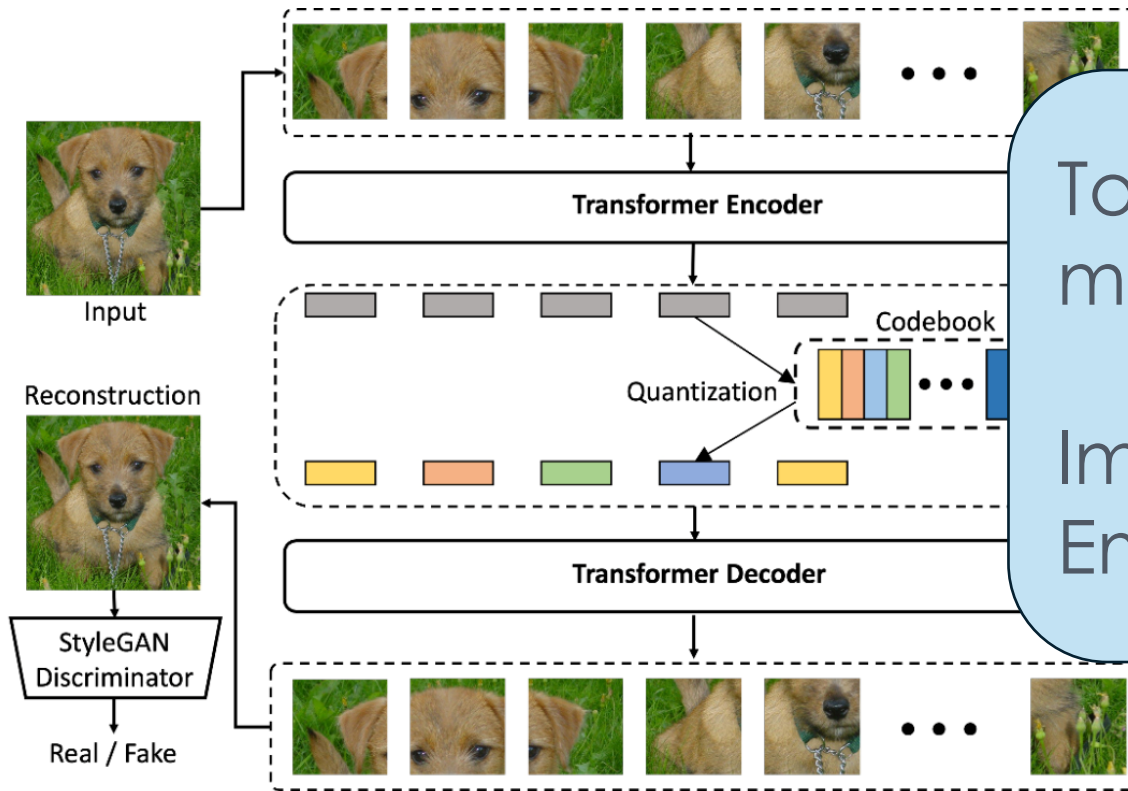
<https://research.google/blog/vector-quantized-image-modeling-with-improved-vqgan/>

Images – patches as token



<https://research.google/blog/vector-quantized-image-modeling-with-improved-vqgan/>

Images – patches as token



Tokenization and embeddings must enable bidirectional mapping

Image $\rightarrow$ patches $\rightarrow$ embeddings
Embeddings $\rightarrow$ patches $\rightarrow$ images

Generation Understanding

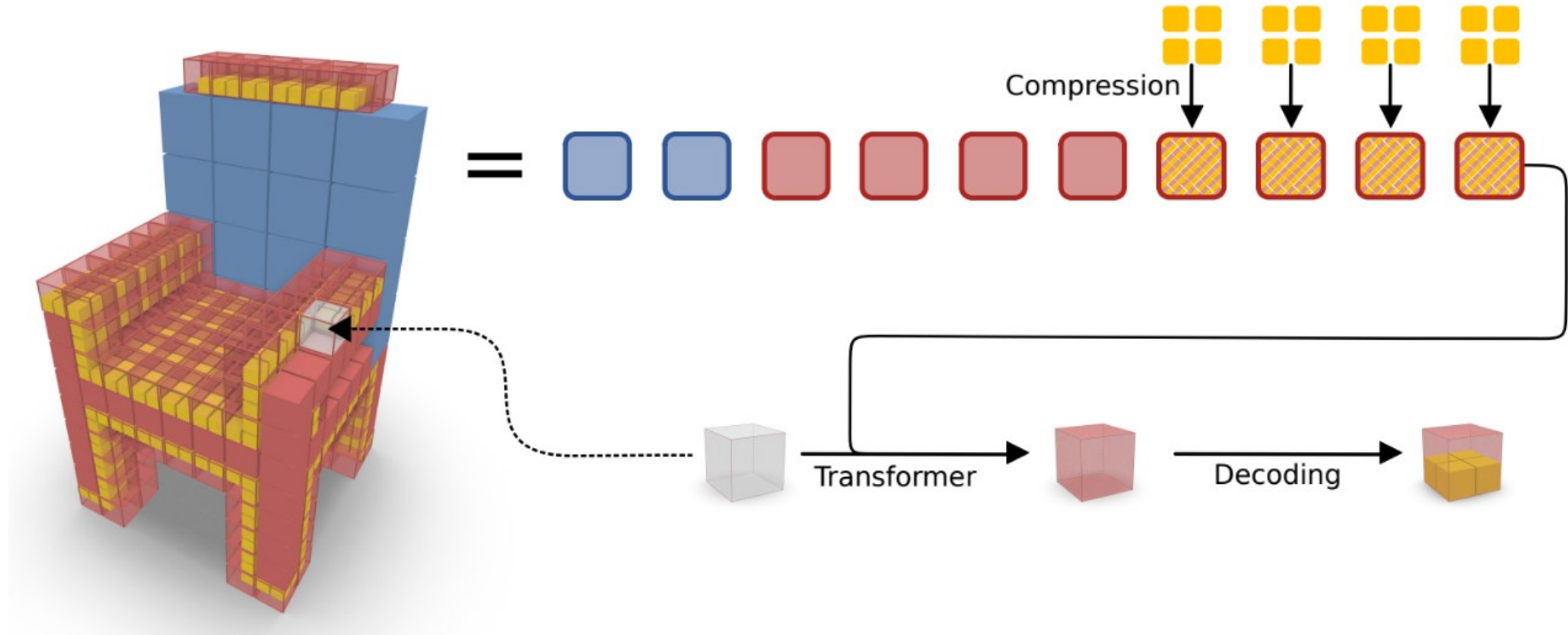
Dog Cat Pig

Stage 1: Image Quantization

Stage 2: Vector-quantized Image Modeling

<https://research.google/blog/vector-quantized-image-modeling-with-improved-vqgan/>

3D – Octrees as token



<https://github.com/GregorKobsik/Octree-Transformer>

Sampling strategies

- LLM sampling strategies control how models select the next token, balancing coherence, creativity, and speed.
- Greedy sampling: pick the token with highest probability
 - issue #1: low variability
 - Issue #2: local optimal selection does not guarantee global maximization
- Solutions:
 - use temperature-scaled softmax (low T more predictable outputs; high T more varied but potentially incoherent ones)

$$p_i = \frac{\exp(z_i/T)}{\sum_{j=1}^V \exp(z_j/T)}$$

Temperature T : scalar parameter that controls the sharpness of the softmax output distribution.

- use alternative sampling, such as Top-K (ranking) selection, beam search, etc.

Major generative model families

Autoregressive Models

GPT, PixelCNN, WaveNet

- Computes exact likelihood
- + Multi-modal, flexible model
- Slow inference (token-by-token)
- Depends on tokenization

Variational Autoencoders

VAE, β -VAE, VQVAE

Latent space

GANs

GAN, StyleGAN, BigGAN

- Implicit distribution
- + Fast inference (single step)
- + Photo-realism
- Unstable training

Normalizing Flows

Glow, RealNVP, MAF

Exact inference

Diffusion Models

DDPM, Stable Diffusion, DALL-E

State of the art

Energy-Based Models

EBM, NCSN, ScoreNet

Flexible density